The Globe oposiciones

Bloque 3 - Tema 6

ARQUITECTURA JAVA EE/JAKARTA EE Y PLATAFORMA .NET: COMPONENTES, PERSISTENCIA Y SEGURIDAD. CARACTERÍSTICAS, ELEMENTOS, LENGUAJES Y FUNCIONES DE AMBOS ENTORNOS. DESARROLLO DE INTERFACES

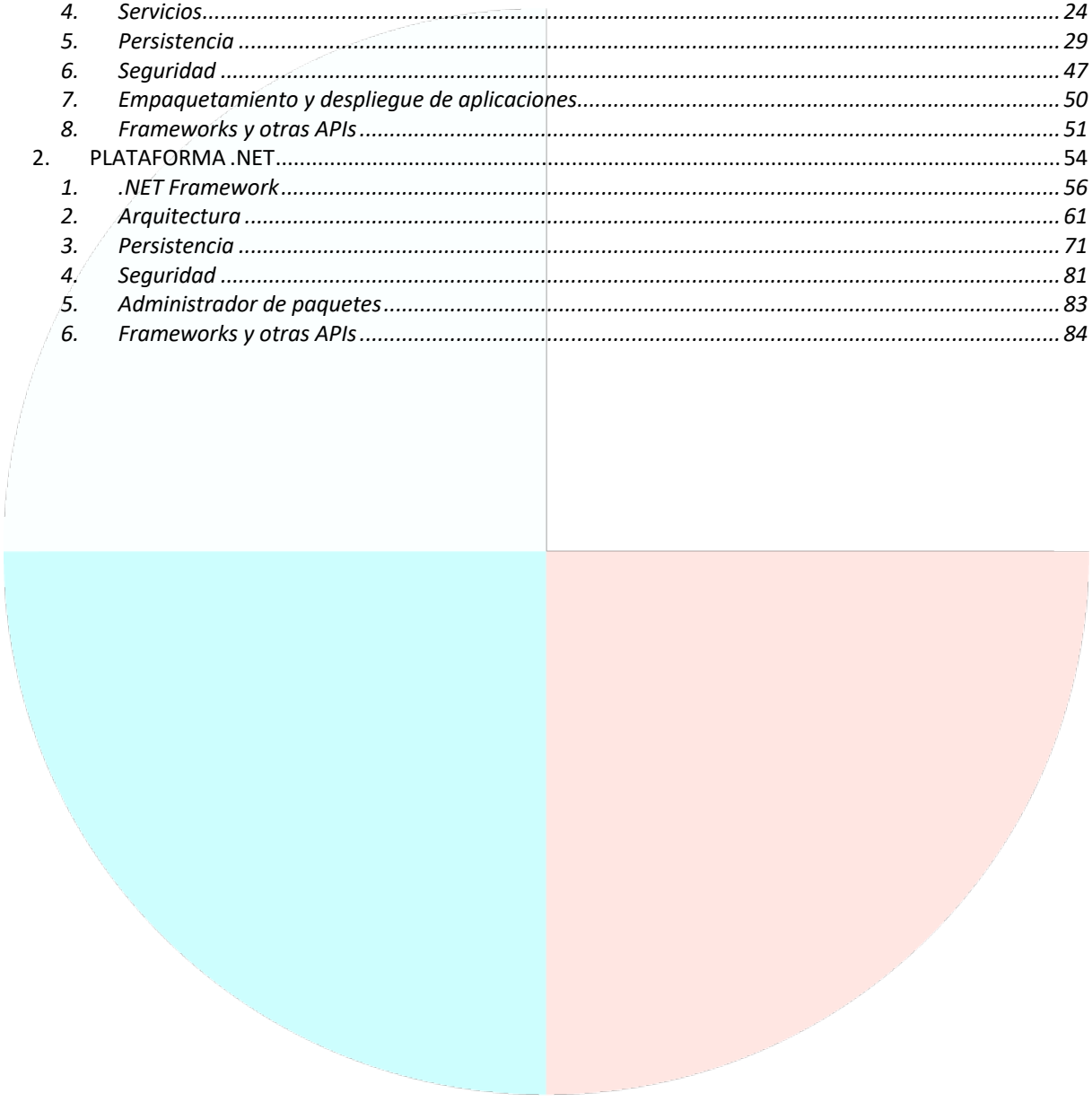
PREPARACIÓN OPOSICIONES

TÉCNICOS AUXILIARES DE INFORMÁTICA

ÍNDICE

ÍNDICE 2

1. ARQUITECTURA JAKARTA EE 3
   1. *Arquitectura 5*
   2. *Servidor Jakarta EE y contenedores 8*
   3. *Componentes 12*
   4. *Servicios 24*
   5. *Persistencia 29*
   6. *Seguridad 47*
   7. *Empaquetamiento y despliegue de aplicaciones 50*
   8. *Frameworks y otras APIs 51*
2. PLATAFORMA .NET 54
   1. *.NET Framework 56*
   2. *Arquitectura 61*
   3. *Persistencia 71*
   4. *Seguridad 81*
   5. *Administrador de paquetes 83*
   6. *Frameworks y otras APIs 84*



1. ARQUITECTURA JAKARTA EE

Las arquitecturas web han revolucionado el panorama del desarrollo de software en los últimos años. Esta revolución se ha debido a diferentes causas de entre las que destacan poderosamente la explosión de Internet y sus protocolos, y el apoyo por parte de la industria a este nuevo paradigma de aplicación. El desarrollo de sistemas corporativos basados en una arquitectura Web es algo relativamente reciente.

Durante los años 90, los desarrollos de aplicaciones corporativas se diseñaban con tecnología cliente/servidor. La implantación de la tecnología cliente/servidor no se hizo sin problemas ya que dieron lugar a entornos heterogéneos y distribuidos que eran muy difíciles de integrar y sobre todo de gestionar.

La necesidad de distribuir aplicaciones a un creciente número de clientes, situados en centros geográficamente dispersos, creó importantes problemas de eficiencia, eficacia y servicio durante la utilización de estas aplicaciones cliente/servidor.

El objetivo inicial de alcanzar flexibilidad, acceso rápido y seguro a los datos corporativos, a un precio razonable, seguía siendo una meta por conseguir. El coste de gestión de estos sistemas era enorme y las corporaciones seguían buscando una nueva tecnología que resolviese al menos algunos de estos problemas.

A finales de los 90, gran número de desarrollos corporativos empezaron a aprovechar los estándares, lenguajes y protocolos surgidos alrededor de la Web, creándose el concepto de las **arquitecturas multicapa**, que se presentaban como alternativa a la arquitectura cliente/servidor clásica, que permitiría utilizar un entorno cliente ligero y, por lo tanto, evitar el problema de la distribución y mantenimiento del software de los PCs cliente.

Indudablemente estos inicios fueron duros ya que la tecnología que se estaba utilizando no había sido inicialmente pensada para lo que ahora se quería utilizar. Las empresas de software iniciaron una carrera para conseguir adaptar los protocolos, lenguajes y estándares de Internet, que parecían claramente asentados en un tipo de aplicaciones con gran cantidad de información estática e interfaces de usuario sencillas, a las necesidades de los desarrollos corporativos estratégicos. Java es uno de estos lenguajes que aparecieron alrededor de Internet y ha impulsado el desarrollo de aplicaciones con arquitectura Web como ningún otro lenguaje.

La complejidad de las arquitecturas web seguía siendo bastante importante, poco a poco se definieron recomendaciones y estándares que intentaron simplificarlo. Con la publicación de las especificaciones J2EE (Java 2 Enterprise Edition) se dio el impulso definitivo para la entrada de esta tecnología en el ámbito empresarial, con muchas menos dudas, ya que estos estándares sí fueron pensados para la problemática de complejos desarrollos corporativos y se ha convertido en la base para los próximos años de la arquitectura de desarrollo para la Web con tecnología Java.

Jakarta Enterprise Edition o **Jakarta EE** es un conjunto de **especificaciones y prácticas** que permiten desarrollar, desplegar y gestionar aplicaciones Java multicapa de entorno empresarial.

La plataforma Jakarta EE consta de un conjunto de servicios, APIs y protocolos que proporcionan la funcionalidad necesaria para desarrollar aplicaciones en lenguaje de programación Java basadas en una arquitectura distribuida de varios niveles o capas.

NOVEDAD

La plataforma Java EE 8 es la última versión especificada por Oracle en el año 2017. A partir de ese momento la organización **Eclipse Foundation** es la nueva encargada. Como Java es una marca registrada por Oracle, en 2018 Eclipse cambió el nombre de Java EE a **Jakarta EE**. Actualmente, versión 11.

Jakarta EE modifica el nombre de los JCP a Eclipse EE.next Working Group (**EE.next**).

La plataforma Java EE se desarrolla a través de Java Community Process (**JCP**), que es responsable de todas las tecnologías Java. Grupos de expertos compuestos por partes interesadas han creado solicitudes de especificación Java (**JSR**) para definir las diversas tecnologías Java EE. El trabajo de la Comunidad Java bajo el programa JCP ayuda a garantizar los estándares de estabilidad y compatibilidad entre plataformas de la tecnología Java.

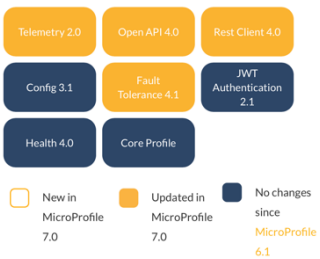
NEW

Las especificaciones Jakarta EE 11 se consideran generales. Tenemos especificaciones de:

- **Plataforma**: incluye un conjunto de especificaciones individuales.
- **Perfil**: solo incluyen aquellas especificaciones individuales para desarrollar un tipo específico de aplicaciones. Perfiles:
  - **Perfil web**.
  - **Perfil core**.



MicroProfile es una especificación de la comunidad de código abierto para microservicios Enterprise Java. Esta comunidad colabora dentro de un grupo de trabajo de la Fundación Eclipse para llevar microservicios a la comunidad Enterprise Java. MicroProfile adopta el perfil Core de Jakarta EE.



Además de la plataforma Jakarta EE, se citan otras plataformas existentes (propiedad de Oracle):

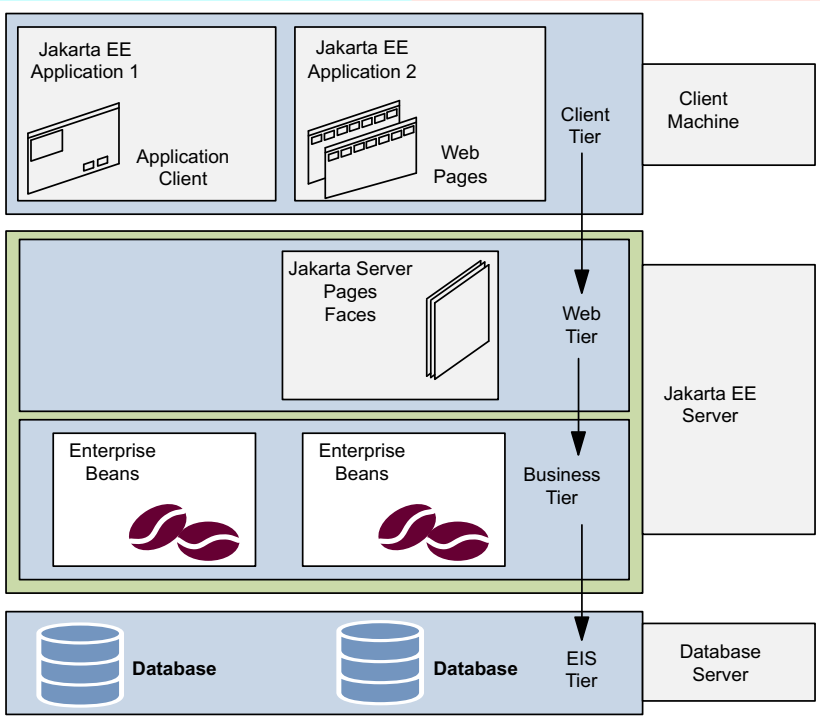
- **Java Card**: permite que elementos seguros, como tarjetas inteligentes y otros chips de seguridad a prueba de manipulaciones, alojen aplicaciones basadas en tecnología Java. Pensado para IoT.
- **Java ME (Micro Edition)**: proporciona un entorno robusto y flexible para aplicaciones que se ejecutan en dispositivos integrados y móviles: microcontroladores, sensores, puertas de enlace, teléfonos móviles, PDAs, decodificadores de TV e impresoras entre otros. Java Embedded es parte de esta plataforma**.**
- **Java SE (Standard Edition)**: permite desarrollar e implementar aplicaciones Java de escritorio y servidor.

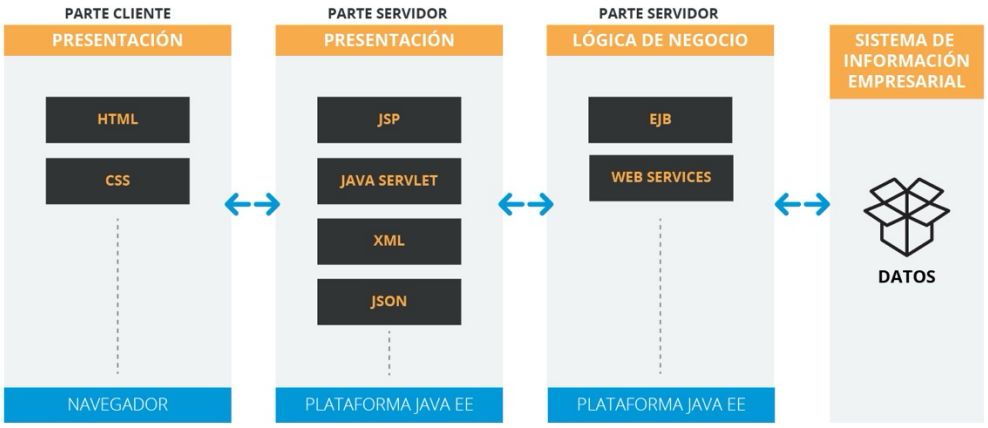
1. Arquitectura

Los desarrollos basados en arquitecturas multicapa, como es el caso de Jakarta EE, utilizan dos conceptos que conviene aclarar:

- **Capa**: una capa es una funcionalidad de la aplicación separada del resto. Por tanto, desde un punto de vista *lógico*, una aplicación se puede dividir en varias capas.
- **Nivel**: un nivel es un entorno físico donde residen una o varias capas. Es un enfoque desde un punto de vista *físico*.

La arquitectura multicapa que sigue capa es la que se muestra en la figura:





Por lo general, las aplicaciones de varias capas tienen una capa de cliente, una capa de negocio y una capa de datos. El nivel de cliente consta de un programa de cliente que realiza solicitudes al nivel de servidor. Éste se divide en una capa web y una capa de lógica de negocio, que manejan las solicitudes de los clientes y procesan los datos de la aplicación, almacenándolos en un almacén de datos permanente en el nivel de datos. El desarrollo de aplicaciones Jakarta EE se concentra en la capa web y de negocio.

La arquitectura de desarrollo multicapa se caracteriza por dividir el sistema en distintas capas que se comunican a través de interfaces:

(1) Nivel de cliente

El nivel de cliente consta de clientes de aplicaciones que acceden a un servidor Jakarta EE y que generalmente se encuentran en una máquina diferente del servidor. Los clientes hacen solicitudes al servidor. El servidor procesa las solicitudes y devuelve una respuesta al cliente. Muchos tipos diferentes de aplicaciones pueden ser clientes Jakarta EE.

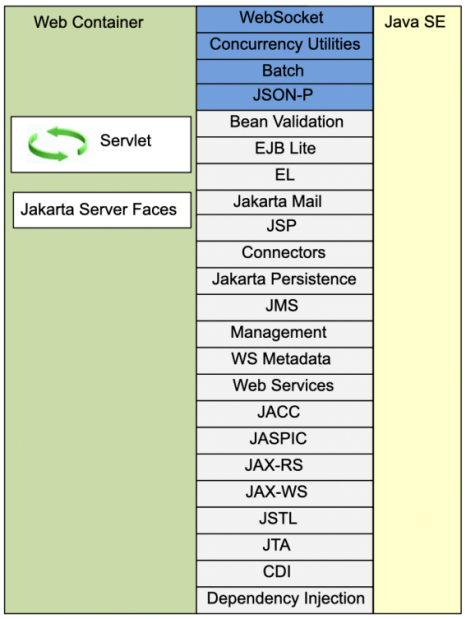
Los clientes pueden ser un **navegador web**, una **aplicación independiente** u otros servidores, y se ejecutan en una máquina diferente del servidor Jakarta EE.

(2) Nivel servidor Jakarta EE - Capa de presentación web

La capa web consta de componentes que manejan la interacción entre los clientes y la capa de negocio. Sus tareas principales son las siguientes:

- Generar dinámicamente contenido en varios formatos para el cliente.
- Recopilar información de los usuarios de la interfaz del cliente y devolver los resultados apropiados de los componentes en el nivel empresarial.
- Controlar el flujo de pantallas o páginas en el cliente.
- Mantener el estado de los datos para la sesión de un usuario.
- Realizar una lógica básica y mantener algunos datos temporalmente en beans administrados.

Algunas de las principales tecnologías Jakarta EE que se utilizan en la capa son (en la imagen, todas las APIs):



- **Jakarta Faces (JSF)**: un marco de componentes de interfaz de usuario para aplicaciones web que le permite incluir componentes de IU (como campos y botones) en una página HTML, llamada página **Facelets**; convierten y validan datos de componentes de la interfaz de usuario, guardan datos de componentes de la IU en almacenes de datos del lado del servidor y mantienen el estado del componente.
- **Jakarta Servlets**: clases del lenguaje de programación Java que procesan dinámicamente solicitudes y construyen respuestas, generalmente para páginas HTML.
- **Jakarta Server Pages (JSP)**: nos permite incrustar en el código HTML código Java para generar las partes dinámicas.

(3) Nivel servidor Jakarta EE - Capa de negocio o lógica de negocio

La capa de negocio consta de componentes que proporcionan la lógica empresarial para una aplicación. La lógica empresarial es un código que proporciona funcionalidad a un dominio comercial particular, como la industria financiera o un sitio de comercio electrónico. En una aplicación empresarial diseñada adecuadamente, la funcionalidad principal existe en los componentes de nivel empresarial.

En esta capa se encuentran las siguientes tecnologías Jakarta EE (en la imagen, todas las APIs):



- **Jakarta RESTful Web Services** (en Java EE, JAX-RS, Java API for RESTful Web Services): API para desarrollo de servicios web RESTful.
- **Jakarta XML Web Services** (en Java EE, JAX-WS, Java API for XML Web Services): provee funcionalidad para servicios web que utilizan mensajes XML y que siguen el estándar SOAP.
- **Jakarta SOAP with Attachments** (en Java EE, SAAJ, SOAP with Attachments API for Java): API que permite la producción y el consumo de mensajes que cumplen con las especificaciones SOAP.

(4) Nivel servidor Jakarta EE - Capa de persistencia o acceso a datos

La capa de persistencia se encarga de la manipulación de los datos que requiere el sistema de información y que están situados en diferentes fuentes de datos. Esta capa puede estar diferenciada de la capa de negocio o estar integrada en ella.

Las tecnologías de Jakarta EE para el acceso a los datos son:

- API de conectividad a base de datos (**JDBC**).
- API para acceso a servicios de directorio (**JNDI**)
- **Jakarta Persistence** (versión actual 3.2). La API de persistencia de Java (en Java EE, JPA).
- **Jakarta Transactions** (versión actual 2.0). La API de transacciones Java (en Java EE, JTA).
- **Jakarta Data** (versión actual 1.0). La API para simplifiar el acceso a los datos.

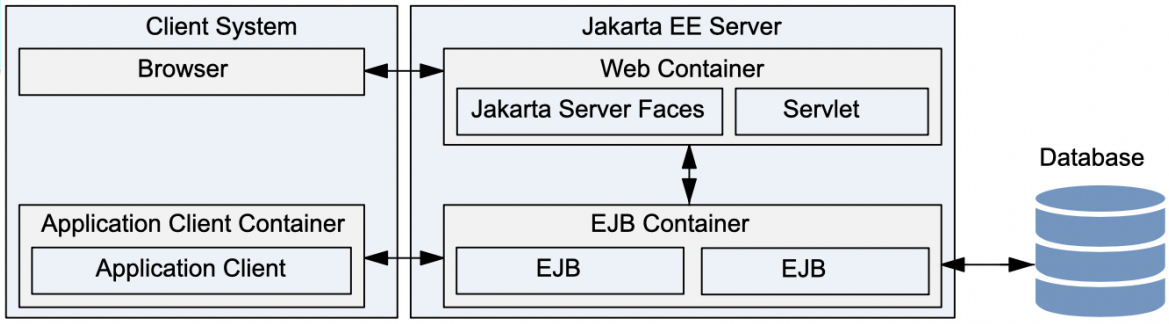
(5) Nivel de datos

Este nivel consta de servidores de bases de datos, sistemas de planificación de recursos empresariales y otras fuentes de datos. Estos recursos generalmente se encuentran en una máquina separada del servidor Jakarta EE, y se accede a ellos por componentes de la lógica de negocio.

2. Servidor Jakarta EE y contenedores

Un servidor Jakarta EE es un **servidor de aplicaciones** que implementa las API de la plataforma Java EE y proporciona servicios estándar Jakarta EE (capa web y de negocio).

Los servidores Jakarta EE que cumplen las especificaciones Jakarta EE se les denomina **servidores de aplicaciones certificados**.



Los servidores de aplicaciones Java EE certificados son productos comerciales:

	PRODUCTO	COMPAÑÍA
Java EE	GlassFish Server Open Source	Código abierto
	Eclipse GlassFish	Eclipse Foundation
	WebSphere	IBM
	WebLogic	Oracle
	JOnAS	ObjectWeb
	JBoss	Red Hat
	TongWeb Application Server	TongTech
	WildFly	WildFly
Jakarta EE 11 PLATFORM y WEB PROFILE	Eclipse GlassFish	Eclipse Foundation
	Server Community	Payara
Jakarta EE 11 CORE PROFILE	Software Enterprise Application Platform	Fujitsu
	Open Liberty	Open Liberty
	Server Community	Payara
	WildFly	WildFly
Jakarta EE 10 PLATFORM	Apusic AAS	Apusic
	BES Application Server	BES
	Eclipse GlassFish	Eclipse Foundation
	Software Enterprise Application Platform	Fujitsu
	WebSphere Liberty	IBM
	InforSuite Application Server	Infors
	Open Liberty	Open Liberty
	Server Community y Server Enterprise	Payara
	JBoss Enterprise Application Platform	Red Hat
	TongWeb Application Server	TongTech
	WildFly	WildFly
Jakarta EE 10 WEB PROFILE y CORE PROFILE	Eclipse GlassFish	Eclipse Foundation
	WebSphere Liberty	IBM
	JBoss Enterprise Application Platform	Red Hat
	Open Liberty	Open Liberty
	Server Community y Server Enterprise	Payara
	WildFly	WildFly

Los servidores Jakarta EE alojan varios tipos de componentes de aplicación que corresponden a las capas en una aplicación multicapa. El servidor Jakarta EE proporciona servicios a estos componentes en forma de contenedor.

Un **contenedor** es un proceso donde se ejecutan los componentes.

Los contenedores Jakarta EE son la interfaz entre el componente y la funcionalidad de la capa inferior proporcionada por la plataforma para soportar ese componente. La funcionalidad del contenedor está definida por la plataforma y es diferente para cada tipo de componente. No obstante, el servidor permite que los diferentes tipos de componentes trabajen juntos para proporcionar funcionalidad en una aplicación empresarial.

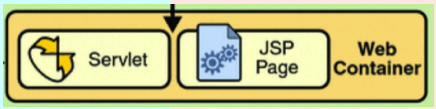




- El **contenedor del cliente** de la aplicación es la interfaz entre los clientes de la aplicación Jakarta EE (aplicaciones especiales de Java SE que usan componentes del servidor Jakarta EE) y el servidor Jakarta EE.

  El contenedor del cliente de la aplicación se ejecuta en la máquina del cliente y es la puerta de enlace entre la aplicación del cliente y los componentes del servidor Jakarta EE que utiliza el cliente.

- El **contenedor web** es la interfaz entre los componentes web y el servidor web. Un componente web puede ser un servlet, una página JSP o JSF. El contenedor gestiona el ciclo de vida del componente, envía solicitudes a los componentes de la aplicación y proporciona interfaces para datos de contexto, como información sobre la solicitud actual. El contenedor web se ejecuta en el servidor web (habitualmente) o en el servidor Jakarta EE.



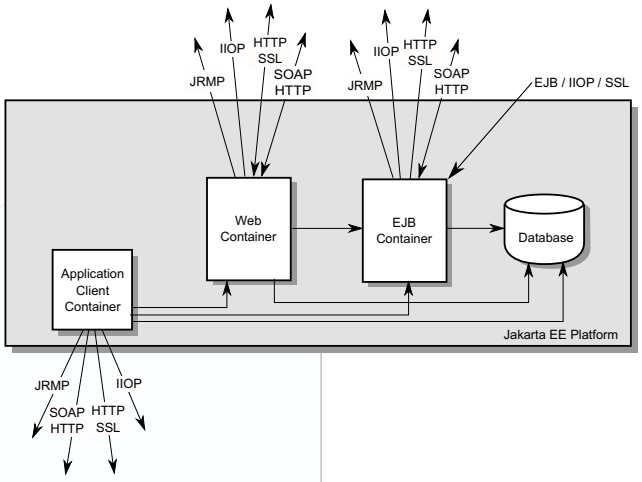
  Contenedores web: Apache Tomcat es un contenedor de servlets y JSPs. El protocolo **AJP** (Apache Jserv Protocol), orientado a paquetes, permite la comunicación entre el servidor web (por ejemplo, Apache) y Tomcat a través de conexiones TCP persistentes. Apache sirve las peticiones de páginas estáticas y Tomcat las peticiones de páginas con contenido dinámico.

- El **contenedor EJB** es la interfaz entre los beans empresariales, que proporciona la lógica empresarial de una aplicación Jakarta EE, y se ubica en el servidor Jakarta EE. El contenedor EJB se ejecuta en el servidor Jakarta EE y gestiona la ejecución de los beans empresariales de una aplicación.

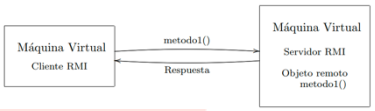


Interoperabilidad

Muchas de las API proveen interoperabilidad con componentes que no forman parte de la plataforma Jakarta EE, como servicios web externos o CORBA. **Jakarta EE Interoperability** ofrece las facilidades de interoperabilidad que pueden estar disponibles en la plataforma Jakarta EE.



Java RMI (RMI, Remote Method Invocation) es una tecnología que proporciona comunicación remota entre programas escritos en el lenguaje Java, permitiendo tener los objetos distribuidos en diversas máquinas, esto es, que un objeto que se ejecuta en una JMV (máquina virtual Java) invoque métodos en un objeto que se ejecuta en otra JVM. Anterior a esta tecnología, para realizar llamadas a funciones y procedimientos remotos se usaba (RPC, Remote Procedure Calls).



Para usar Java RMI se necesita el siguiente paquete:

```
import java.rmi.*;
```

Cada servicio RMI (objeto remoto) se define mediante una interface, que especifica los métodos que se pueden invocar en el objeto remoto. Esta interface debe estar disponible en el cliente y en el servidor. Además, se puedan pasar argumentos al método remoto y recibir los datos que devuelve (tipos primitivos u objetos de clases que sean serializables).

RMI-JRMP es el protocolo de invocación de método remoto (RMI, Remote Method Invocation) específico de Java que permite la comunicación de aplicaciones distribuidas basadas exclusivamente en Java.

Java IDL (IDL, Interface Description Language) es una implementación CORBA que permite que dos objetos interactúen sobre diferentes plataformas a través de una red. Al ser una interfaz, posibilita que objetos escritos con diferentes lenguajes (a diferencia de Java RMI) interactúen entre sí.

RMI/IIOP (IIO, Internet Inter-ORB Protocol) es un protocolo de comunicación para CORBA que utiliza RMI para permitir la comunicación entre aplicaciones distribuidas en diferentes lenguajes (no necesariamente Java).

3. Componentes

Los **componentes** se despliegan y ejecutan en contenedores especializados.

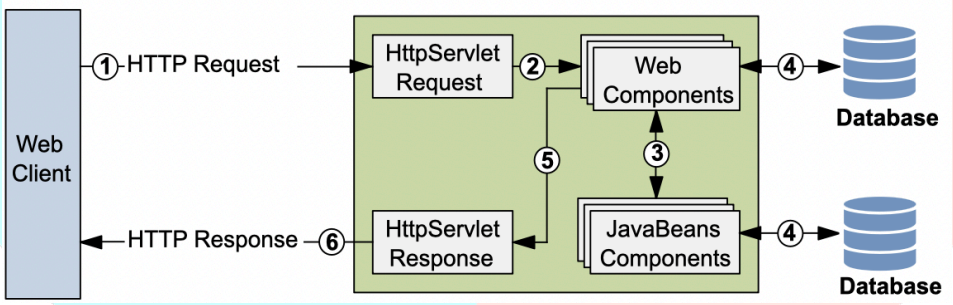
Citamos los siguientes componentes:

- Componentes **WEB**:
  - Jakarta **Servlet**. Versión actual 6.1. (en Java EE, **Java Servlet**)
  - Jakarta Pages (**JSP**). Versión actual 4.0. (en Java EE, **JavaServer Pages**)
  - Jakarta Faces (**JSF**). Versión actual 4.1. (en Java EE, **JavaServer Faces**)
- Componentes de **NEGOCIO**:
  - Jakarta Enterprise Beans (**EJB**). Versión actual 4.0. (en Java EE, **Enterprise JavaBeans**)

(1) Servlets

Un servlet es una clase de lenguaje de programación Java que se utiliza para ampliar las capacidades de los servidores que alojan aplicaciones a las que se accede mediante un modelo de programación de petición/respuesta. Se utilizan en los servidores web proporcionando contenido dinámico al usuario. Para este tipo de aplicaciones, la tecnología Java Servlet define clases de servlet específicas de HTTP.

Los servlets son pequeños programas escritos en Java que admiten peticiones a través del protocolo HTTP. Por lo tanto, los servlets pueden recibir peticiones desde un navegador web, procesarlas, y devolver una respuesta al navegador, normalmente en HTML. Para realizar estas tareas podrán utilizar las clases incluidas en el lenguaje Java.

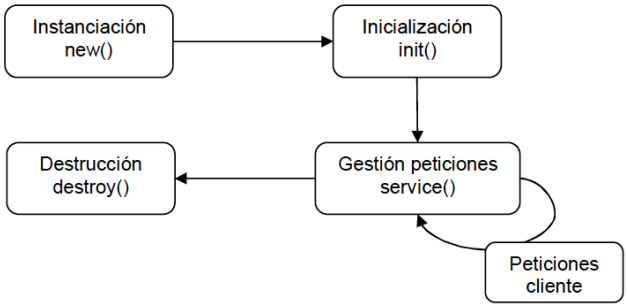


Los servlets se instalan, gestionan y ejecutan en un componente del sistema denominado contenedor web, que se comunican con el cliente normalmente mediante el protocolo HTTP. Estos programas suelen ser intermediarios entre el cliente (la mayoría de las veces un navegador web) y los datos (normalmente una base de datos).

El **funcionamiento** básico de un servlet es el siguiente:

- El cliente envía una petición al servlet mediante HTTP.
- El servlet procesa los argumentos de la petición.
- El servlet realiza la tarea solicitada.
- El servlet devuelve una respuesta al cliente, normalmente en HTML.

El **ciclo de vida** de un servlet tiene 3 fases:



- **Instanciación e inicialización**: se crea el servlet en memoria (si no existe una instancia previa del mismo) y se llama al método `init()` para inicializar su funcionamiento.
- **Gestión de peticiones**: para atender cada petición, el servidor de aplicaciones o contenedor de servlets crea un hilo de ejecución del servlet, lo que permite atender varias peticiones de forma simultánea, compartir datos de forma automática, etc. Como desventaja, es importante tener en cuenta los posibles problemas de concurrencia que pueden derivarse de esto.
- **Destrucción**: si durante un tiempo (configurable) el servlet no recibe peticiones, es descargado de memoria.

La estructura básica de un servlet comprende:

- La importación de las clases necesarias para desarrollar nuestros propios servlets, son:

```
import jakarta.servlet.*; (en Java EE, javax.servlet.*)
import jakarta.servlet.http.*; (en Java EE, javax.servlet.http.*)
```

- La clase a implementar debe extender un servlet del API. Como las aplicaciones a desarrollar son aplicaciones web utilizaremos como clase base el servlet `HttpServlet`, que implementa el comportamiento de un servlet capaz de trabajar con el protocolo http:

```
public class MiClase extends HttpServlet { ... }
```

  La clase `HttpServlet` es una clase abstracta de la que heredarán todos los servlets destinados a dar servicios desde un contenedor web. Los métodos más interesantes y que un servlet puede sobrecargar son los siguientes:

  - `doGet()`: se ejecuta cuando se recibe una petición HTTP GET.
  - `doPost()`: se ejecuta cuando se recibe una petición HTTP POST.
  - `doPut()`: se ejecuta cuando se recibe una petición HTTP PUT.
  - `doDelete()`: se ejecuta cuando se recibe una petición HTTP DELETE.
  - `init()` y `destroy()`: gestionan el ciclo de vida del servlet, ejecutándose en la creación y destrucción del servlet.
  - `getServletInfo()`: lo utiliza el servlet para proporcionar información acerca de sí mismo.
  - `service()`: maneja la petición HTTP para redirigir al método `doXXX` que corresponda.

  Los métodos `doXXX` reciben como parámetros, objetos de la clase `HttpServletRequest` con la información de la solicitud HTTP y `HttpServletResponse`, donde se incluye la información de respuesta a la petición HTTP. En ellos se implementa normalmente la funcionalidad del servlet ante los distintos tipos de solicitudes HTTP.

- Después debemos sobrescribir, si lo necesitamos, el método `init()` heredado de la superclase, encargado de inicializar el servlet. En él debemos incluir todo lo que queramos que se ejecute sólo cuando se cree el servlet en memoria. Para ello, primero llamamos al `init()` de la superclase. Después podemos incluir las inicializaciones necesarias.

```
public void init(ServletConfig conf) throws ServletException {
	super.init(conf);
	// Inicializaciones
}
```

- Finalmente se sobrescriben los métodos `doXXX` según los tipos de peticiones que se vayan a tratar.

Ejemplo:

```
<html>
	<body>
		<form action="ArgumentosPost" method="post">
			<label for="nombre">Nombre<label/>
			<input type="text" name="nombre" size="25" maxlength="50"><br/>
			<label for="apellidos">Apellidos<label/>
			<input type="text" name="apellidos"
				size="35" maxlength="100"><br/>
			<input type="submit" value="Enviar"/>
			<input type="reset" value="Borrar"/>
		</form>
	</body>
</html>
```

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ArgumentosPost extends HttpServlet {
	public void doPost(HttpServletRequest req, HttpServletResponse res)
			throws ServletException, IOException {
		res.setContentType("text/plain");
		PrintWriter out = res.getWriter();
		out.println("Nombre: " + req.getParameter("nombre"));
		out.println("Apellidos: " + req.getParameter("apellidos"));
	}
}
```

El protocolo HTTP es un protocolo que no es capaz de almacenar el estado en el que se encuentra, es decir, no es capaz de almacenar qué ha pasado en el pasado inmediato. Los servidores de aplicaciones suplen esa carencia permitiendo que almacenemos variables en un entorno de ejecución de nuestra aplicación web denominado **sesión**. Los servlets son capaces de conectarse a una determinada sesión en curso o crear una nueva y añadir, modificar o borrar atributos de la sesión.

Ejemplo: aplicación web que cuenta el número de veces que accedemos o recargamos una página mientras la sesión está activa.

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class Contador extends HttpServlet {
	public void doGet(HttpServletRequest request, HttpServletResponse response)
			throws ServletException, IOException {
		PrintWriter out = response.getWriter();
		HttpSession session = request.getSession(true);
		Integer contador;
		if (session.isNew()) {
			out.println("Hola. ");
			contador = new Integer(0);
		} else {
			out.println("Hola de nuevo. ");
			contador = (Integer)session.getAttribute("valor");
		}
		Integer nuevoValor = new Integer(contador.intValue()+1);
		session.setAttribute("valor", nuevoValor);
		out.println("En la sesión: "+session.getId()+
			" el contador vale " + contador + "\n");
	}
}
```

(2) Faces

Jakarta Faces es un framework IU para aplicaciones web Java. Está diseñado para reducir significativamente la carga de escribir y mantener aplicaciones que se ejecutan en un servidor de aplicaciones Java. Jakarta Faces:

- Facilita la construcción de IU a partir de un conjunto de componentes de interfaz de usuario reutilizables.
- Simplifica la migración de los datos de la aplicación hacia y desde la IU.
- Ayuda a gestionar el estado de la IU en todas las solicitudes del servidor.
- Proporciona un modelo sencillo para comunicar eventos generados por el cliente al servidor.
- Permite crear y reutilizar fácilmente los componentes de IU personalizados.

Para usar Jakarta Faces se necesita el siguiente paquete:

```
import jakarta.faces.*;
```
(en Java EE, `javax.faces`)

Java Server Faces (JSF) ofrece:

- Un conjunto de APIs para representar componentes de IU y administrar su estado, manejar eventos y validación de entradas, definir la navegación de la página y respaldar la internacionalización y la accesibilidad.
- Una biblioteca de etiquetas personalizadas de JSP para expresar una interfaz JSF dentro de una página JSP.

La página web myfacelet.xhtml está construida utilizando etiquetas de componente JSF, las cuales se utilizan para agregar componentes a la vista (myView), que es la representación de

la página en el lado del servidor. La página web también puede hacer referencia a objetos, como eventos, validadores y conversores que estén registrados en los componentes, y también a los componentes JavaBeans que capturan los datos y procesan la funcionalidad específica de la aplicación de los componentes.

A petición del cliente, la vista se presenta como respuesta. La renderización es el proceso mediante el cual, basándose en la vista del lado del servidor, el contenedor web genera una salida HTML que puede ser leída por el navegador.

Los componentes de JSF son:

- **Componentes IU**: clases que extienden de IUComponentBase (subclase de UIComponent), la cual define el estado y el comportamiento por defecto de los componentes. Componentes: UIColumn, UICommand, IUForm, UIInput, UIMessage, UIPanel…
- **Conversor**: transforma de forma conveniente los valores de los objetos Java vinculados a los componentes JSF en Strings y transforman los Strings enviados por el cliente (parámetros HTTP) en los objetos Java vinculados a los componentes JSF según el tipo que corresponda.
- **Validador**: comprueba que los valores recibidos y almacenados en los componentes JSF cumplen las restricciones especificadas.
- **Renderizador**: genera la representación de los componentes a mostrar en los clientes y de recuperar las entradas de usuario recibidas para actualizar los valores vinculados a los componentes.

Por último, recordamos que tanto Jakarta Faces como JSF son una especificación, por lo que necesitamos de un framework que implemente dicha especificación. Entre las implementaciones destacamos:

- **Eclipse MojarraTM** (incluido en Eclipse GlassFish) es una implementación que define un marco MVC para crear IU para aplicaciones web, incluidos los componentes de la IU, la gestión del estado, la entrega de eventos, la validación de entradas, la navegación por páginas y el soporte para la internacionalización y la accesibilidad.
- **Apache MyFaces**.

(3) JSP

JSP es un lenguaje que permite insertar etiquetas de distintos tipos que añaden comportamiento dinámico a nuestras páginas. JSP se considera un nivel de abstracción por encima de los servlets, ya que las páginas JSP se transforman en servlets para ejecutarse.

Ejemplo:

```
<%@page import="java.util.*"%>
<%! String cadena="Hola Mundo!!!"; %>
<html>
	<body>
		<%= cadena %>
```

```
	</body>
</html>
```

Salvo por algunos detalles, el código incluido entre las etiquetas <%...%> es **código Java** normal y corriente: podemos definir variables, mostrar su contenido, debemos incluir paquetes, etc.

Se puede incluir código Java dentro de las páginas JSP de 3 formas distintas:

- **Expresiones**: que se evalúan y el resultado se muestra en el navegador. Pueden ser tan sencillas como <%= cadena %> incluida en el ejemplo anterior o pueden contener concatenaciones y acceso a atributos estáticos de clases de java.

  Ejemplo:

```
<%@page import="java.util.*"%>
<html>
	<body>
		<%= "El valor del número Pi es: "+Math.PI %>
	</body>
</html>
```

- **Scriptlets**: son fragmentos de código Java entre las etiquetas <% ... %>. Para poder mostrar cosas en el navegador debemos utilizar el objeto implícito out.

  Ejemplo:

```
<%@page import="java.util.*"%>
<html>
	<body>
<%
	out.println("<table border=\"1\">");
	for(int i='a', j=0; i<='z'; i++, j++) {
		out.println("<tr>");
		out.println("<td>"+j+"</td>");
		out.println("<td>"+(char)i+"</td>");
		out.println("</tr>");
	}
	out.println("</table>");
%>
	</body>
</html>
```

- **Declaraciones**: que se encuentran entre los símbolos <%! ... %>. Las variables declaradas se consideran globales. En el ejemplo “hola mundo”, podemos ver cómo se declara una variable global de tipo String que luego se utiliza.

  Ejemplo:

```
<%! String cadena="Hola Mundo!!!"; %>
```

Como se comentó en la introducción, los archivos JSP se transforman en un servlet para su ejecución. Las **directivas** influyen en la manera en la que se realiza esta transformación. Hay tres tipos de directivas:

- `<%@page ... %>`: se utiliza para importar clases, acceder a la sesión, gestión de errores, etc.

- `<%@include ... %>`: nos permite incluir un fichero dentro de otro. Simplemente copia el contenido de un fichero dentro de otro.

- `<%@taglib ... %>`: se utiliza en el uso de JSTL[1] (JavaServer Pages Standard Tag Library).

Las **acciones** tienen diversos usos, como por ejemplo incluir la salida resultante de ejecutar un fichero JSP en otro. Podemos escribir un fichero index.jsp de la forma.

Ejemplo:

```
<html>
	<body>
		<jsp:include page="cuerpo.jsp"/>
	</body>
</html>
```

Y un fichero cuerpo.jsp de la forma:

```
<%
	for(int i=0; i<5; i++) {
		out.println("Hola Mundo!!!");
	}
%>
```

En JSP existen los denominados **objetos implícitos**. Estos objetos no necesitan ser instanciados y nos permitirán realizar multitud de acciones. Normalmente serán objetos que en los servlets se recibían como argumentos de los métodos.

- **request**: Es un objeto de tipo `HttpServletRequest` que nos ofrece, fundamentalmente, acceso a los parámetros de la petición realizada al servlet.

  Ejemplo: el formulario, al no tener definido el método ni el lugar donde enviar los argumentos para su procesamiento, envía el parámetro a la propia página.

```
<html>
	<body>
		<form>
		   <input type="text" name="parametro"/>
		   <input type="submit"/>
		</form>
		<p>
		   Valor del parámetro:<%= request.getParameter("parametro") %>
		</p>
	</body>
</html>
```

[1] Extiende JSP proporcionando cuatro bibliotecas de etiquetas: core, xml, sql y fmt.

- **response**: objeto de tipo `HttpServletResponse` que ofrece métodos para procesar la respuesta de la página JSP, trabajar con cookies, etc.

- **out**: el objeto `out`, instancia de la clase `JspWriter`, permite escribir la salida que recibirá el navegador.

- **session**: objeto de la clase `HttpSession` que nos permite trabajar con sesiones.

- **config**: este objeto de la clase `ServletConfig` nos da acceso a los parámetros de configuración del servlet. La configuración de una página JSP se realiza de la misma forma que un servlet, ya que en su ejecución se transforman en uno.

La **gestión de errores** en páginas JSP se realiza mediante excepciones. JSP incluye dos directivas para el tratamiento de excepciones: `errorPage` y `isErrorPage`. Para indicar en una página JSP que página se va a encargar de tratar la excepción debemos utilizar la directiva:

```
<%@page errorPage="..." %>
```

Ejemplo: Si el parámetro es un número no hay problema, pero si se para algo que no pueda transformarse en un entero se creará una excepción y nos redirige a la página de errores.

```
<%@page errorPage="error.html" %>
<html>
	<body>
		<p>
		Número: <%= Integer.parseInt(request.getParameter("numero")) %>
		</p>
	</body>
</html>
```

El único problema es que podemos mostrar una nueva web en caso de error, pero no podemos trabajar con la excepción generada. Para eso debemos utilizar `isErrorPage`:

```
<%@page isErrorPage = "true"%>
```

Ejemplo:

```
<%@ page isErrorPage = "true"%>
<% if (exception != null) {
	exception.printStackTrace(new java.io.PrintWriter(out));
} %>
```

Para trabajar con **sesiones** lo primero es indicar en la página web que se va a usar una sesión:

```
<%@page session="true" %>
```

Después se trabajará con el objeto implícito `session`, que nos permitirá añadir y eliminar atributos de la sesión. También nos permitirá eliminar la sesión.

Ejemplo:

```
<%@page session="true" %>
<%
```

```
        Integer accesos = (Integer)session.getAttribute("accesos");
        if (accesos==null)
              accesos=new Integer(0);
        accesos = new Integer(accesos.intValue()+1);
        session.setAttribute("accesos", accesos);
        if (request.getParameter("invalidaSesion")!=null)
              session.invalidate();
%>
<html>
        <body>
            <form>
              <input type="submit" name="invalidaSesion" value="Invalidar sesión"/>
              <input type="submit" value="Recargar página"/>
            </form>
              <p>Contador: <%= accesos.intValue() %> </p>
        </body>
</html>
```

Por último, al definir una página JSP nos interesa indicar mediante la directiva `@page` la codificación de la propia página y tipo de contenido del resultado de la ejecución de dicha página. Para ello, utilizamos:

```
<%@page pageEncoding="UTF-8" contentType="text/html;charset=UTF-8"%>
```

(4) EJB

La arquitectura se define en Jakarta Enterprise Beans. Escrito en el lenguaje de programación Java, un Enterprise Java Bean es, por tanto, una clase, un componente del lado del servidor que encapsula la lógica de negocios de una aplicación. La lógica de negocios es el código que cumple el propósito de la aplicación.

Ejemplo: en una aplicación de control de inventario, los beans pueden implementar la lógica de negocio en métodos llamados `checkInventoryLevel` y `orderProduct`. Al invocar estos métodos, los clientes pueden acceder a los servicios de inventario proporcionados por la aplicación.

Debemos considerar el uso de EJB si la aplicación tiene alguno de los siguientes requisitos:

- La aplicación debe ser escalable. Para dar cabida a un número creciente de usuarios, es posible que haya que distribuir los componentes de una aplicación en varias máquinas. Los beans empresariales de una aplicación no solo pueden ejecutarse en diferentes máquinas, sino que también su ubicación permanecerá transparente para los clientes.
- Las transacciones deben garantizar la integridad de los datos. Los beans admiten transacciones, los mecanismos que administran el acceso concurrente de objetos compartidos.
- La aplicación tendrá una variedad de clientes. Con solo unas pocas líneas de código, los clientes remotos pueden localizar fácilmente beans empresariales. Estos clientes pueden ser diversos y numerosos.

Para crear un EJB hacemos uso del paquete:

```
import jakarta.ejb.*;
```
(en Java EE, `javax.ejb`)

Existen varios tipos EJB:

- **Bean de SESIÓN (session)**: realiza una tarea para un cliente. Los beans de sesión son de 3 tipos: con estado (**stateful**), sin estado (**stateless**) y **singleton**.

  - **Stateful (con estado)**: el estado de un objeto consiste en los valores de sus variables de instancia. En un bean de sesión con estado, las variables de instancia representan el estado de una sesión única del bean. Debido a que el cliente interactúa ("habla") con su bean, este estado a menudo se denomina estado de conversación.

    Como su nombre lo indica, un bean de sesión es similar a una sesión interactiva. Un bean de sesión no se comparte, solo puede tener un cliente, de la misma manera que una sesión interactiva solo puede tener un usuario. Cuando el cliente finaliza, su bean de sesión ya no está asociado con el cliente.

    Ejemplo: bean de sesión con estado para implementar un carrito.

```
import cart.util.BookException;
import java.util.List;
import Jakarta.ejb.Remote;

@Remote
public interface Cart {
    public void initialize(String person) throws BookException;
    public void initialize(String person, String id)
                        throws BookException;
    public void addBook(String title);
    public void removeBook(String title) throws BookException;
    public List<String> getContents();
    public void remove();
}

import java.io.Serializable;
import java.util.ArrayList;
import java.util.List;
import cart.util.BookException;
import cart.util.IdVerifier;
import jakarta.ejb.Remove;
import jakarta.ejb.Stateful;

@Stateful
public class CartBean implements Cart {
    String customerId;
    String customerName;
    List<String> contents;

    @Override
    public void initialize(String person) throws BookException {
        if (person == null) {
            throw new BookException("Null person not allowed.");
        } else {
```

```
            customerName = person;
        }
        customerId = "0";
        contents = new ArrayList<>();
    }

    @Override
    public void initialize(String person, String id)
                throws BookException {
        if (person == null) {
            throw new BookException("Null person not allowed.");
        } else {
            customerName = person;
        }

        IdVerifier idChecker = new IdVerifier();
        if (idChecker.validate(id)) {
            customerId = id;
        } else {
            throw new BookException("Invalid id: " + id);
        }

        contents = new ArrayList<>();
    }

    @Override
    public void addBook(String title) {
        contents.add(title);
    }

    @Override
    public void removeBook(String title) throws BookException {
        boolean result = contents.remove(title);
        if (result == false) {
            throw new BookException("\"" + title + " not in cart.");
        }
    }

    @Override
    public List<String> getContents() {
        return contents;
    }

    @Remove
    @Override
    public void remove() {
        contents = null;
    }
}
```

Desde la clase CartClient se podrían hacer las siguientes invocaciones de métodos:

```
cart.initialize("Pablo Arellano", "123");
cart.addBook("Bel Canto");
List<String> bookList = cart.getContents();
cart.removeBook("Pensando en Java");
```

- **Stateless (sin estado)**: un bean de sesión sin estado no mantiene un estado de conversación con el cliente. Cuando un cliente invoca los métodos de un bean sin estado, las variables de instancia del bean pueden contener un estado específico para ese cliente pero solo durante la duración de la invocación. Cuando finaliza el método, el estado específico del cliente no debe conservarse. Excepto durante la invocación del método, todas las instancias de un bean sin estado son equivalentes, lo que permite que el contenedor EJB asigne una instancia a cualquier cliente. Es decir, el estado de un bean de sesión sin estado debe aplicarse a todos los clientes.

  Debido a que pueden admitir múltiples clientes, los beans de sesión sin estado pueden ofrecer una mejor escalabilidad para aplicaciones que requieren un gran número de clientes. Por lo general, una aplicación requiere menos beans de sesión sin estado que beans de sesión con estado para admitir la misma cantidad de clientes.

  Un bean de sesión sin estado puede implementar un servicio web, pero un bean de sesión con estado no puede.

  Ejemplo: bean de sesión sin estado para implementar un conversor de divisas.

```
import java.math.BigDecimal;
import jakarta.ejb.*;

@Stateless // Anotaciones
public class ConverterBean {
    private BigDecimal yenRate = new BigDecimal("108.393");
    private BigDecimal euroRate = new BigDecimal("0.00842769");

    public BigDecimal dollarToYen(BigDecimal dollars) {
        BigDecimal result = dollars.multiply(yenRate);
        return result.setScale(2, BigDecimal.ROUND_UP);
    }

    public BigDecimal yenToEuro(BigDecimal yen) {
        BigDecimal result = yen.multiply(euroRate);
        return result.setScale(2, BigDecimal.ROUND_UP);
    }
}
```

- **Singleton**: un bean de sesión singleton se instancia una vez por aplicación y existe para el ciclo de vida de la aplicación. Los beans de sesión Singleton están diseñados para circunstancias en las que una única instancia de Enterprise Bean es compartida y accedida simultáneamente por los clientes.

  Los beans de sesión Singleton ofrecen una funcionalidad similar a los beans de sesión sin estado, pero difieren de ellos en que solo hay un bean de sesión Singleton por aplicación, a diferencia de un grupo de beans de sesión sin estado, cualquiera de los cuales puede responder a una solicitud del cliente. Al igual que los beans de sesión sin estado, los beans de sesión singleton pueden implementar puntos finales de servicio web.

Los beans de sesión Singleton mantienen su estado entre invocaciones de clientes, pero no están obligados a mantener su estado en los bloqueos o apagados del servidor.

Las aplicaciones que usan un bean de sesión singleton pueden especificar que el singleton se debe instanciar al inicio de la aplicación, lo que permite que el singleton realice tareas de inicialización para la aplicación. El singleton también puede realizar tareas de limpieza en el cierre de la aplicación, porque el singleton operará durante todo el ciclo de vida de la aplicación.

Ejemplo: bean de sesión Singleton para implementar un contador.

```
import jakarta.ejb.Singleton;
/**
 * CounterBean is a simple singleton session bean that records the number
 * of hits to a web page.
 */
@Singleton
public class CounterBean {
    private int hits = 1;

    // Increment and return the number of hits
    public int getHits() {
        return hits++;
    }
}
```

- **Bean dirigido POR MENSAJES (message-driven)**: permite que un componente de negocio puede recibir mensajes asíncronamente.

  Un bean controlado por mensajes es un bean que permite a las aplicaciones Jakarta EE procesar mensajes de forma asincrónica. Este tipo de bean normalmente actúa como un listener de mensajes JMS, que es similar a un listener de eventos pero recibe mensajes JMS en lugar de eventos. Los mensajes pueden ser enviados por cualquier componente Jakarta EE (un cliente de aplicación, otro bean empresarial o un componente web) o por una aplicación o sistema JMS que no utiliza la tecnología Jakarta EE. Los beans controlados por mensajes pueden procesar mensajes JMS u otros tipos de mensajes.

- **Bean de ENTIDAD (entity)**: (opcional) es un objeto de entidad que forma parte del modelo de dominio, proporcionando una vista de los datos de la base de datos.

4. Servicios

En este apartado se relacionan las APIs y servicios de la arquitectura:

- **Jakarta Activation** (en Java EE, JAF, JavaBeans Activation Framework): define un conjunto de servicios estándar para determinar el tipo de datos, encapsular el acceso a ellos, descubrir las operaciones disponibles en ellos y crear el componente JavaBeans apropiado para realizar esas operaciones.

- **Jakarta Annotations**: colección de anotaciones que representan conceptos semánticos comunes que permiten un estilo declarativo de programación que se aplica en una variedad de tecnologías Java.

- **Jakarta Batch**: especifica una API de Java más un lenguaje de especificación de trabajos basado en XML, que permite componer trabajos por lotes en XML a partir de artefactos de aplicaciones Java reutilizables y parametrizar convenientemente diferentes ejecuciones en un solo trabajo.

- **Jakarta Concurrency**: proporciona una especificación para usar la concurrencia de los componentes de la aplicación sin comprometer la integridad del contenedor y al mismo tiempo preservar los beneficios fundamentales de la plataforma Jakarta EE.

- **Jakarta Config**: permite leer datos de configuración de diferentes fuentes compatibles con el entorno de forma portátil.

- **Jakarta Context Dependency Injection (CDI)**: inyección de dependencia declarativa y servicios de apoyo.

- **Jakarta Dependency Injection**: anotaciones de inyección de dependencia declarativa común.

- **Jakarta Debugging Support for Other Languages**: mecanismo mediante el cual los programas ejecutados en la JVM pero escritos en lenguajes distintos a Java se pueden depurar con referencias a la fuente original (por ejemplo, referencias de archivos fuente y números de línea).

- **Jakarta Deployment**: API de despliegue para aplicaciones Jakarta EE ensambladas.

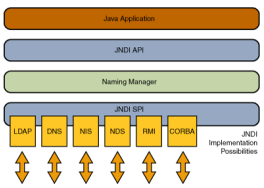
- **Jakarta Interceptors**: define un medio de interposición en invocaciones de métodos comerciales y eventos específicos, como eventos del ciclo de vida y eventos de tiempo de espera, que ocurren en instancias de componentes de Jakarta EE y otras clases administradas.

- **Jakarta Management**: define un modelo de gestión estándar para exponer y acceder a la información de gestión, las operaciones y los parámetros de los componentes de la plataforma Jakarta EE.

- **Jakarta MVC**: define un estándar para la creación de aplicaciones web siguiendo el patrón MVC.

- **Jakarta WebSocket**: proporciona soporte para crear aplicaciones de WebSocket. WebSocket es un protocolo de aplicación que proporciona comunicaciones full-duplex entre dos pares a través del protocolo TCP.

- **JNDI** (Java Naming and Directory Interface): API para servicios de directorio o nombrado (LDAP, DNS, RMI, CORBA, sistema de ficheros…).



Capa de presentación

- **Jakarta Faces**.

- **Jakarta Pages**.

- **Jakarta Servlet**.

- **Jakarta Expression Language** (EL): define un lenguaje de expresión para aplicaciones Java.

- **Jakarta Standard Tag Library** (en Java EE, JSTL, JavaServer Pages Standard Tag Library): bibliotecas de etiquetas que encapsula como etiquetas simples la funcionalidad principal común a muchas aplicaciones web. JSTL tiene soporte para tareas comunes y estructurales como iteración y condicionales, etiquetas para manipular documentos XML, etiquetas de internacionalización y etiquetas SQL.

Capa de negocio

- **Jakarta Enterprise Beans**.

  Jakarta Managed Beans: define un conjunto de servicios básicos para objetos gestionados por contenedores con requisitos mínimos, conocidos como POJOs.

- **Jakarta Bean Validation**: define un modelo de metadatos y API para Java Bean y validación de métodos.

- **Jakarta Validation** (actualmente en desarrollo, sustituirá a Jakarta Bean Validation): proporciona una función de declaración y validación de restricciones a nivel de objeto para el desarrollador de aplicaciones Java, así como un repositorio de metadatos de restricciones y una API de consulta.

Mensajes y comunicaciones

- **Jakarta Mail** (en Java EE, JavaMail): API para enviar notificaciones por correo electrónico.

- **Jakarta Messaging** (en Java EE, JMS, Java Messages Service): API que permite a las aplicaciones crear, enviar, recibir y leer mensajes. Define un conjunto común de interfaces y semánticas asociadas que permiten que los programas escritos en lenguaje Java se comuniquen con otras implementaciones de mensajería.

JSON

- **Jakarta JSON Binding** (JSON-B): proporciona una capa de enlace para convertir objetos Java hacia y desde mensajes JSON.

- **Jakarta JSON Processing** (JSON-P): API para analizar, transformar y consultar datos JSON.

Persistencia

- **Jakarta Data**: ofrece acceso a datos, como bases de datos relacionales y no relacionales, servicios de datos basados en la nube…

- **Jakarta NoSQL**: especificación independiente con el objetivo de integrarse con bases de datos NoSQL.

- **Jakarta Persistence** (en Java EE, JPA, Java Persistence API): proporciona una función de mapeo de objetos/relacionales para administrar datos relacionales en aplicaciones Java.

- **Jakarta Transactions** (en Java EE, JTA, Java Transaction API): proporciona una interfaz estándar para delimitar las transacciones.

- **JDBC** (Java Database Connectivity): API que permite acceder a las bases de datos utilizando objetos.

Interoperabilidad

- **Jakarta Connectors** (en Java EE, **Java EE Connector**): define una arquitectura estándar para conectar la plataforma Jakarta EE a sistemas de información empresarial (ERP) heterogéneos.

- **Jakarta RPC**: define un estándar para usar gRPC.

- **Jakarta XML RPC** (en Java EE, JAX-RPC, Java API for XML-based RPC): permite la llamada a procedimientos remotos utilizando un protocolo basado en XML, es decir, la conexión de un programa Java con un web service aunque éste no esté desarrollado en Java.

Seguridad

- **Jakarta Security** (en Java EE, Java EE Security): define un estándar para crear aplicaciones seguras de Jakarta EE en paradigmas de aplicaciones modernos.

- **Jakarta Authentication** (en Java EE, JASPIC, Java Authentication Service Provider Interface for Containers): define una interfaz de proveedor de servicios (SPI) mediante la cual los proveedores de autenticación que implementan mecanismos de autenticación de mensajes pueden integrarse en contenedores.

- **Jakarta Authorization** (en Java EE, JACC, Java Authorization Contract for Containers): especificación que define un contrato entre un servidor de aplicaciones Jakarta EE y un proveedor de políticas de autorización.

- **JAAS** (Java Authentication and Authorization Service): es un conjunto de APIs que permiten que los servicios autentiquen y apliquen controles de acceso a los usuarios. JAAS proporciona un marco conectable y extensible para la autenticación y autorización programática de usuarios.

Servicios web

- **Jakarta Enterprise Web Services**: define los servicios web para la arquitectura de Jakarta EE.

- **Jakarta Web Services Metadata**: define la sintaxis y la semántica de los metadatos y valores predeterminados de Java Web Service.

- **Jakarta XML Registries** (en Java EE, JAXR, Java API for XML Registries): encargada de facilitar el acceso a UDDI para acceder a los registros de negocio.

- **Jakarta RESTful Web Services** (en Java EE, JAX-RS, Java API for RESTful Web Services): API para desarrollo de servicios web RESTful.

- **Jakarta XML Web Services** (en Java EE, JAX-WS, Java API for XML Web Services): provee funcionalidad para servicios web que utilizan mensajes XML y que siguen el estándar SOAP.

- **Jakarta SOAP with Attachments** (en Java EE, SAAJ, SOAP with Attachments API for Java): API que permite la producción y el consumo de mensajes que cumplen con las especificaciones SOAP.

Procesamiento XML

- **Jakarta XML Binding** (en Java EE, JAXB, Java Architecture for XML Binding): proporciona una API y herramientas que automatizan el mapeo entre documentos XML y objetos Java.

- **JAXP** (Java API for XML Processing): admite el procesamiento de documentos XML mediante el modelo DOM o el modelo SAX (API simple para XML). Esta API permite que las aplicaciones analicen y transformen documentos XML independientemente de una implementación particular de procesamiento de XML.

  El **modelo SAX** procesa los documentos en serie (hacia delante), convirtiendo los elementos de un documento XML en una serie de eventos. Cada elemento del documento XML genera un evento concreto. El desarrollador proporciona los métodos manejadores para esos eventos que realizarán las acciones deseadas. El procesamiento con SAX es más rápido por su modo de acceso y menor utilización de memoria.

  El **modelo DOM** de procesamiento de documentos XML genera un árbol a partir del documento. Aunque requiere mucha más memoria para almacenar el árbol, hace que la codificación sea más sencilla.

- **StAX** (Streaming API for XML): alternativa a SAX y DOM para el procesamiento de documentos XML.

5. Persistencia

En la capa de lógica de negocio tendremos una "subcapa" dedicada al acceso de los datos ubicados en el nivel de datos. La subcapa de acceso a los datos se puede llevar a cabo de 2 formas diferentes:

- **JDBC**: como ya se ha comentado, las aplicaciones pueden utilizar el API JDBC para acceder a los datos en un sistema de gestión de bases de datos relacionales (RDBMS).
- **ORM**: mediante el mapeo objeto/relacional a través de frameworks que implementen JPA.
- **JNDI**: se utiliza este API para acceder a datos en directorios, tipo LDAP.

JDBC

Para conectarnos con una base datos y trabajar con ella hacemos uso de los paquetes:

```
import java.sql.*;
```

Clase / Interface	Función
DriverManager	Establece la conexión con la base de datos
Connection	Representa una conexión con la BD
Statement	Ejecución de consultas SQL (**CRUD** "Crear, Leer, Actualizar y Borrar")
PreparedStatement	Almacena en un objeto la compilación de una instrucción SQL, para ser ejecutada muchas veces de manera eficiente
ResultSet	Manipulación de registros en consultas de tipo select

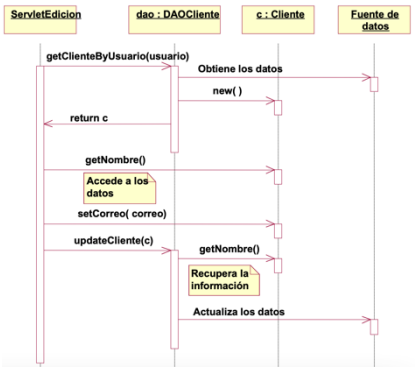
```
import javax.sql.*;
```

Clase / Interface	Función
DataSource	Información para la conexión con la base de datos

Un objeto `DataSource` tiene un conjunto de propiedades que identifican y describen el origen de datos del mundo real que representa. Estas propiedades incluyen información como la ubicación del servidor de la base de datos, el nombre de la base de datos, el protocolo de red que se utilizará para comunicarse con el servidor, etc.

Adicionalmente se necesita un **driver JDBC** que es una implementación de dichas interfaces:

- Específico para cada programa gestor de base de datos.
- Proporcionado habitualmente por el proveedor del gestor de base de datos.



Utilizaremos un **Data Access Object** (DAO) para abstraer y encapsular todos los accesos a la fuente de datos. El DAO maneja la conexión con la fuente de datos para obtener y almacenar datos. El DAO oculta completamente los detalles de implementación de la fuente de datos a sus clientes. Como la interfaz expuesta por el DAO no cambia cuando cambia la implementación de la fuente de datos subyacente, este patrón permite al DAO adaptarse a diferentes esquemas de almacenamiento sin que esto afecte a sus clientes o componentes de negocio. Esencialmente, el DAO actúa como un adaptador entre el componente y la fuente de datos.

Así pues, para clase del modelo (por ejemplo: persona, venta, factura, expediente, profesor, alumno…) crearemos un DAO específico.

Ejemplo: si tenemos un sistema que trabaja con alumnos, comentamos las clases a definir para acceder a los datos de asignaturas. El sistema gestor de base de datos es MySQL. En BD, se supone creada la tabla `asignaturas`:

```
CREATE TABLE asignaturas (
	idasignatura int(11) NOT NULL AUTO_INCREMENT,
	nombre varchar(100),
	PRIMARY KEY (idasignatura) );
```

Clase `Asignatura`: clase del modelo, es la clase POJO.

```
public class Asignatura {
	private int idAsignatura;
	private String nombre;

	public Asignatura() {}

	public Asignatura(int idAsignatura, String nombre) {
		this.idAsignatura = idAsignatura;
		this.nombre = nombre;
	}

	public int getIdAsignatura() { return idAsignatura; }

	public void setIdAsignatura(int idAsignatura) {
		this.idAsignatura = idAsignatura;
	}

	public String getNombre() { return nombre; }

	public void setNombre(String nombre) { this.nombre = nombre; }

	@Override
	public String toString() {
		return idAsignatura + " - " + nombre;
	}
}
```

Clase `AsignaturaDAO`: clase de servicio que se encarga del acceso y manipulación de las asignaturas en la base datos.

```
import java.util.ArrayList;
import java.sql.*;

public class AsignaturaDAO {
	private Connection conexion;
	private String driver = "com.mysql.jdbc.Driver";
	private String url = "jdbc:mysql://localhost:3306/";
	private String nombreBD = "academia";
	private String user = "pruebas";
	private String pass = "pruebas";

	public AsignaturaDAO {
		try {

			Class.forName(driver);
			conexion = DriverManager.getConnection(url + nombreBD,
							user, pass);
		} catch (SQLException e) {
			e.printStackTrace();
		}
	}

	public void add(String nombre) {
		Statement stmt = conexion.createStatement();
		String insert= "INSERT INTO asignaturas(nombre) VALUES(\"" + nombre +
		"\");");
		stmt.executeUpdate(insert);
	}

	public void update(Asignatura a) {
		Statement stmt = conexion.createStatement();
		String update= "UPDATE asignaturas SET nombre = \"" + a.getNombre() +
		"\" WHERE idasignatura = " +
		a.getIdAsignatura() + ";";
		stmt.executeUpdate(update);
	}

	public void delete(Asignatura a) {
		Statement stmt = conexion.createStatement();
		String delete= "DELETE FROM asignaturas WHERE idasignatura = +
		a.getIdAsignatura() + ";";
		stmt.executeUpdate(delete);
	}

	public Asignatura getAsignatura(int id) {
		Statement stmt = conexion.createStatement();
		String select = "SELECT * FROM asignaturas WHERE idasignatura = " +
		a.getIdAsignatura() + ";";
		ResultSet rs = stmt.executeQuery(select);
		Asignatura aux;

		if (rs.next()) {
			aux = new Asignatura(rs.getInt(1), rs.getString(2)));
		}

		return aux;
	}

	public ArrayList<Asignatura> list(String nombre) {
		Statement stmt = conexion.createStatement();
```

```
            String select = "SELECT * FROM asignaturas WHERE nombre LIKE \"" +
            nombre + "\";";
            ResultSet rs = stmt.executeQuery(select);
            ArrayList<Asignatura> lista = new ArrayList<Asignatura>();

            while (rs.next()) {
                  lista.add(new Asignatura(rs.getInt(1),
                              rs.getString(2)));
            }

            return lista;
      }
}
```

Con lo visto hasta ahora, el desarrollo nos vincula a una fuente de datos concreta, MySQL. Sin embargo, podemos generalizar e independizar (desacoplar) el uso de un DAO en función de la fuente de datos. Esto significa, que podemos tener almacenados los datos en fuentes de datos diferentes y que afecta a nuestra aplicación el cambio. Así publicamos una interfaz genérica de acceso a los datos (InterfazDAO) que especifique los servicios que se pueden consumir y, por otro lado, creamos una implementación concreta de acceso a los datos (ImplementacionDAOmysql, ImplementacionDAOsqlserver…) para cada fuente de datos diferente.

Ejemplo: aplicamos la independencia deseada al ejemplo anterior.

Clase `Asignatura`: clase del modelo. No varía respecto al ejemplo anterior.

```
public class Asignatura {
      private int idAsignatura;
      private String nombre;

      public Asignatura() {}

      public Asignatura(int idAsignatura, String nombre) {
           this.idAsignatura = idAsignatura;
           this.nombre = nombre;
      }

      public int getIdAsignatura() { return idAsignatura; }

      public void setIdAsignatura(int idAsignatura) {
            this.idAsignatura = idAsignatura;
      }

      public String getNombre() { return nombre; }

      public void setNombre(String nombre) { this.nombre = nombre; }

      @Override
      public String toString() {
            return idAsignatura + " - " + nombre;
      }
}
```

Interfaz `IAsignaturaDAO`: interfaz con las declaraciones de los métodos para poder manipular objetos Asignatura.

```
import java.util.ArrayList;

public interface IAsignaturaDAO {
	public void add(String nombre);
	public void update(Asignatura a);
	public void delete(Asignatura a);
	public Asignatura getAsignatura(int id);
	public ArrayList<Asignatura> list(String nombre);
}
```

Clase `AsignaturaDAOMySqlImpl`: clase de servicio que implementa la interfaz IAsignaturaDAO para acceder a los datos en MySQL.

```
import java.util.ArrayList;
import java.sql.*;

public class AsignaturaDAOMySqlImpl implements IAsignaturaDAO {
	private Connection conexion;
	private String driver = "com.mysql.jdbc.Driver";
	private String url = "jdbc:mysql://localhost:3306/";
	private String nombreBD = "academia";
	private String user = "pruebas";
	private String pass = "pruebas";

	public AsignaturaDAOMySqlImpl() {
		//Igual que constructor AsignaturaDAO
		try {

			Class.forName(driver);
			conexion = DriverManager.getConnection(url + nombreBD,
					user, pass);

		} catch (SQLException e) {
			e.printStackTrace();}

	}

	public void add(String nombre) {
		//Igual que método add AsignaturaDAO
		Statement stmt = conexion.createStatement();
		String insert= "INSERT INTO asignaturas(nombre) VALUES(\"" + nombre +
		"\");");
		stmt.executeUpdate(insert);
	}

	public void update(Asignatura a) {
		//Igual que método update AsignaturaDAO
		Statement stmt = conexion.createStatement();
		String update= "UPDATE asignaturas SET nombre = \"" + a.getNombre() +
		"\" WHERE idasignatura = " +
		a.getIdAsignatura() + ";";
		stmt.executeUpdate(update);
	}

	public void delete(Asignatura a) {
		//Igual que método delete AsignaturaDAO
		Statement stmt = conexion.createStatement();
		String delete= "DELETE FROM asignaturas WHERE idasignatura = +
		a.getIdAsignatura() + ";";
		stmt.executeUpdate(delete);
	}
```

```
	public Asignatura getAsignatura(int id) {
		//Igual que método getAsignatura AsignaturaDAO
		Statement stmt = conexion.createStatement();
		String select = "SELECT * FROM asignaturas WHERE idasignatura = " +
		a.getIdAsignatura() + ";";
		ResultSet rs = stmt.executeQuery(select);
		Asignatura aux;

		if (rs.next()) {
			aux = new Asignatura(rs.getInt(1), rs.getString(2)));
		}

		return aux;
	}

	public ArrayList<Asignatura> list(String nombre) {
		//Igual que método list AsignaturaDAO
		Statement stmt = conexion.createStatement();
		String select = "SELECT * FROM asignaturas WHERE nombre LIKE \"" +
		nombre + "\";";
		ResultSet rs = stmt.executeQuery(select);
		ArrayList<Asignatura> lista = new ArrayList<Asignatura>();

		while (rs.next()) {
			lista.add(new Asignatura(rs.getInt(1), rs.getString(2)));
		}
		return lista;
	}
}
```

En el caso de que cambie la fuente de datos bastará con crear una nueva clase de servicio `AsignaturaDAONuevaFuenteImpl` para el acceso concreto al nuevo sistema de base de datos que implemente la interfaz `IAsignaturaDAO`.

```
import java.util.ArrayList;
import java.sql.*;

public class AsignaturaDAOOracleImpl implements IAsignaturaDAO {...}
	private Connection conexion;
	private String driver = "oracle.jdbc.driver.OracleDriver";
	private String url = "jdbc:oracle:thin:@localhost:1521:oracl";
	private String user = "pruebas";
	private String pass = "pruebas";

	public AsignaturaDAOOracleImpl {
		try {
			Class.forName(driver);
			conexion = DriverManager.getConnection(url, user, pass);
		} catch (SQLException e) {
			e.printStackTrace();
		}
	}
	// Resto igual que la implementación para MySQL
}
```

Vemos ahora un ejemplo de uso sencillo para le gestión de asignaturas.

```
import java.util.ArrayList;
```

```
public class Prueba {
	public static void main(String[] args){
		IAsignaturaDAO dao = new AsignaturaDAOMySQLImpl();
		//Añadimos varias asignaturas a la base de datos
		dao.add("Inteligencia artificial");
		dao.add("Desarrollo web");
		dao.add("Vision artificial");
		dao.add("Programacion orientada a objetos");
		//Obtener asignatura con id=2
		Asignatura a = dao.getAsigatura(2);
		System.out.println(a);
		//Actualizar asignatura a
		a.setNombre("Desarrollo web I");
		dao.update(a);
		//Eliminar asignatura 2
		dao.delete(a);
		//Mostrar todas las asignaturas guardadas
		ArrayList<Asignatura> lista = dao.list("%”);
		for(Asignatura aux in lista)
			System.out.println(aux);
	}
}
```

Notar que para cambiar el proveedor de datos basta con modificar una línea de código

```
		IAsignaturaDAO dao = new AsignaturaDAOMySQLImpl();
```

por

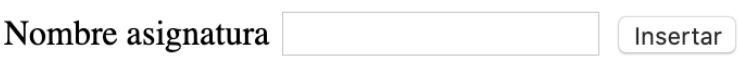
```
		IAsignaturaDAO dao = new AsignaturaDAOOracleImpl();
```

Y ya operamos con la nueva base de datos.

Vamos a desarrollar ahora un ejemplo sencillo con un servlet. Presuponemos tener instalados Tomcat y MySQL. El ejemplo consiste en una página web con un formulario para insertar una nueva asignatura. Cuando el usuario cumplimente el campo nombre y pulse el botón Insertar un servlet se encargará tratar la petición, es decir, de guardar la asignatura en base de datos.

`Formulario.html`: página HTML con formulario para insertar asignatura

```
<html>
    <body>
      <form action="AddAsignatura" method=”POST”>
		<label for="name">Nombre asignatura</label>
		<input type="text" name="nombre"/><br/>
		<input type="submit" value="Insertar"/>
      </form>
    </body>
</html>
```



`AddAsignatura.java`: servlet que guarda la asignatura en base de datos.

```
import java.io.*;
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
public class AddAsignatura extends HttpServlet {
       public void doPost(HttpServletRequest request,
                          HttpServletResponse response)
                          throws ServletException, IOException {
              PrintWriter out = response.getWriter();
              IAsignaturaDAO dao = new AsignaturaDAOMySQLImpl();
              dao.add(request.getParemeter("nombre"));
              out.println("La asignatura se ha guardado correctamente!");
```

La asignatura se ha guardado correctamente!

ORM

Jakarta Persistence (**JPA**) es una especificación que proporciona a los desarrolladores de Java un recurso de **mapeo de objeto/relacional** (ORM) para administrar datos relacionales en aplicaciones Java mediante el uso de objetos regulares, denominados POJOs.

A través del fichero **persistence.xml** se establece la conexión con la base de datos y se definen las entidades que se gestionarán (unidad de persistencia).

Un **POJO** (Plain Old Java Object) es una clase simple que tiene atributos privados, constructores y métodos getters y setters para devolver los valores de los atributos o para asignarles un valor. Para persistir una clase ésta debe cumplir una serie de requisitos:

- Debe tener un constructor público sin argumentos.
- Cada uno de los atributos a persistir debe tener un método get y otro set asociado.
- Debe implementar la interfaz `Serializable` (paquete **`java.io`**).

Podemos usar el paquete de persistencia con:

```
import jakarta.persistence.*;
```

Una **entidad** es un objeto de dominio de persistencia ligero. Típicamente, una entidad representa una tabla en una base de datos relacional, y cada instancia de entidad corresponde a una fila en esa tabla. El artefacto de programación principal de una entidad es la clase de entidad, aunque las entidades pueden usar clases auxiliares. De esta forma, una clase de entidad es un POJO.

Ejemplo: clase POJO para modelar una empresa.

```
import java.io.Serializable;

public class Empresa implements Serializable {
    private Integer idEmpresa;
    private String cif;
    private String nombre;
    private String administrador;
    private String telefono;
    private String observaciones;
    private Date fechaAlta;
```

```
    public Empresa() { }

    public Empresa(Integer idEmpresa, String cif, String nombre,
             String administrador, String telefono, String observaciones) {
        this.idEmpresa = idEmpresa;
        this.cif = cif;
        this.nombre = nombre;
        this.administrador = administrador;
        this.telefono = telefono;
        this.observaciones = observaciones;
    }

    public Integer getIdEmpresa() { return idEmpresa; }
    public void setIdEmpresa(Integer idEmpresa){this.idEmpresa = idEmpresa;}
    public String getCif() { return cif;  }
    public void setCif(String cif) { this.cif = cif;  }
    public String getNombre() {  return nombre;  }
    public void setNombre(String nombre) { this.nombre = nombre;  }
    public String getAdministrador() { return administrador; }
    public void setAdministrador(String administrador) {
             this.administrador = administrador; }
    public String getTelefono() { return telefono;  }
    public void setTelefono(String telefono) { this.telefono = telefono; }
    public String getObservaciones() { return observaciones; }
    public void setObservaciones(String observaciones) {
             this.observaciones = observaciones; }
    public Date getFechaAlta() { return fechaAlta; }
    public void setFechaAlta(Date fechaAlta) { this.fechaAlta = fechaAlta; }
    @Override
    public String toString() {return "(" + idEmpresa + ") " + cif + nombre;}
}
```

Podemos establecer una **relación** entre una tabla y una clase POJO y entre un registro de la tabla y un objeto de la clase POJO.

El estado persistente de una entidad se representa mediante campos o propiedades persistentes. Estos campos o propiedades usan **anotaciones** de mapeo objeto/relacional o un **documento XML** para mapear las entidades y las relaciones de entidad con los datos relacionales en el almacén de datos subyacente.

En Java solucionamos problemas de negocio a través de objetos, los cuales tienen estado y comportamiento. Sin embargo, las bases de datos relacionales almacenan la información mediante tablas, filas, y columnas, de manera que para almacenar un objeto hay que realizar una correlación entre el sistema orientado a objetos de Java y el sistema relacional de nuestra base de datos. JPA es una abstracción sobre JDBC que nos permite realizar dicha correlación de forma sencilla, realizando por nosotros toda la conversión entre nuestros objetos y las tablas de una base de datos. Esta conversión se llama ORM (Object Relational Mapping, Mapeo Relacional de Objetos), y puede configurarse a través de metadatos mediante:

- **Documento XML**: se genera un fichero para clase entidad indicando la tabla con la que se corresponde la clase, la correlación entre cada una de las variables miembro y los

atributos de la tabla y con las peculiaridades (clave primaria, valor autogenerado…) de cada atributo.

- **Anotaciones**: el resultado es igual que en el caso anterior, pero sin necesidad de crear un fichero XML, ya que las correspondencias se establecen en la propia definición de la clase.

  Para los ejemplos de este apartado, tendremos presente la tabla asignaturas:

```
CREATE TABLE asignaturas (
   idasignatura int(11) NOT NULL AUTO_INCREMENT,
   nombre varchar(100),
   PRIMARY KEY (idasignatura) );
```

  Las principales anotaciones son:

  - `@Entity`: clase que es una entidad a persistir.

```
@Entity
public class Asignatura implements Serializable
```

  - `@Table`: nombre de la tabla donde se persisten los datos de la clase. Es opcional si el nombre de la clase coincide con el de la tabla. Contiene el atributo **`name`** para especificar el nombre de la tabla.

```
@Entity
@Table(name="asignaturas")
public class Asignatura implements Serializable
```

  - `@Id`: atributo de la clase que es clave primaria.

```
@Id
private int idAsignatura;
```

  - `@Column`: mapeo de un atributo de la clase con una columna de la tabla en la que se persistirá. Es opcional si el nombre del atributo Java coincide con el de la columna a persistir. Contiene el atributo **`name`** para especificar el nombre de la columna. El atributo puede ser de un tipo primitivo, String, Date, BigDecimal, BigInteger, un tipo colección o un tipo que implemente la interface Serializable.

```
@Id
@Column(name="idasignatura")
private int idAsignatura;

@Column(name="nombre")
private String nombre;

private Collection<Alumno> alumnos;
```

  - `@GeneratedValue`: especifica para el atributo la estrategia de generación de valores. Contiene el atributo **`strategy`** para establecer el tipo concreto de generación de valores, pudiendo ser AUTO, IDENTITY, SEQUENCE y TABLE.

```
@Id
@GeneratedValue(strategy=GenerationType.AUTO)
@Column(name="idasignatura")
```

```
private int idAsignatura;
```

- **@OneToOne**: para establecer una relación de uno a uno entre clases. Dispone de los atributos:
  - **cascade**: tipo de operación de cascada a realizar. Los valores posibles que admite el enumerador `CascadeType` son REMOVE, REFRESH, PERSIST, MERGE, DETACH y ALL.
  - **fetch**: forma en que se consultarán las entidades asociadas. El enumerador `FetchType` admite los valores EAGER y LAZY.
- **@OneToMany**: para establecer una relación de uno a muchos entre clases. Dispone de los atributos:
  - **mappedBy**: contiene el nombre de la propiedad Java de la clase hija (en el extremo muchos) que enlaza con la clase padre (en el extremo uno).
  - **cascade**: tipo de operación de cascada a realizar.
- **@ManyToOne**: para establecer una relación de muchos a uno entre clases. Esta anotación es acompañada de la anotación **@JoinColumn** en la que se indica mediante el atributo **name** el nombre de la columna que en la tabla hija contiene la clave ajena de la tabla padre.
- **@ManyToMany**: para establecer una relación de muchos a muchos entre clases. Se acompaña con la anotación **@JoinTable** que especifica datos sobre la tabla de unión en la relación muchos a muchos, con los siguientes atributos:
  - **name**: nombre de la tabla.
  - **joinColumns**: columnas que forman parte de la clave primaria que estamos definiendo. Utilizamos para ello una anotación **@JoinColumn** y se indica con el atributo `name` el nombre de la columna.
  - **inverseJoinColumns**: contiene cada una de las columnas que forman la clave primaria de la clase con la que se tiene la relación. Cada columna se indica mediante una anotación **@JoinColumn** y en el atributo `name` se indica el nombre de la columna.

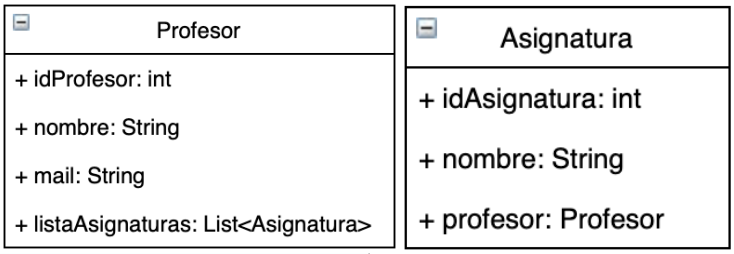
  Si la relación es bidireccional, se debe especificar el extremo propietario mediante el parámetro **mappedBy**.

Ejemplo: Clases con anotaciones relación uno a muchos. Una asignatura es impartida por un profesor y un profesor puede impartir una o varias asignaturas. Se establece una relación uno a muchos entre profesores y asignaturas.
A nivel de base de datos tenemos 2 tablas:

```
CREATE TABLE asignaturas (
      idasignatura int(11) NOT NULL AUTO_INCREMENT,
      nombre varchar(100) NOT NULL,
      idprofesor varchar(100),
      PRIMARY KEY (idasignatura),
      FOREIGN KEY idprofesor REFERENCES profesores;
```

```
CREATE TABLE profesores (
	idprofesor int(11) NOT NULL AUTO_INCREMENT,
	nombre varchar(100) NOT NULL,
	mail varchar(100),
	PRIMARY KEY (idprofesor) );
```

A nivel de clases tenemos:




```
import java.util.List;
import jakarta.persistence.*;

@Entity
@Table(name="profesores")
public class Profesor implements Serializable {
	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	@Column(name="idprofesor")
	private int idProfesor;

	@Column(name="nombre")
	private String nombre;

	@Column(name="mail")
	private String mail;

	@OneToMany(mappedBy="profesor" cascade="CascadeType.ALL)
	private List<Asignatura> listaAsignaturas;

	public Profesor() {}
	public Profesor(String nombre, String mail) {
		this.nombre = nombre;
		this.mail = mail;}
	public int getIdProfesor() { return idProfesor; }
	public void setIdProfesor(int idProfesor) {this.idProfesor = idProfesor;}
	public String getNombre() {return nombre;}
	public void setNombre(String nombre) {this.nombre = nombre;}
	public String getMail() {return mail;}
	public void setMail(String mail) {this.mail = mail;}
	public List<Asignatura> getListaAsignaturas() {return listaAsignaturas;}
	public void setListaAsignaturas(List<Asignatura> a) {listaAsignaturas = a;}
}
import jakarta.persistence.*;

@Entity
@Table(name="asignaturas")
public class Asignatura implements Serializable {

	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	@Column(name="idasignatura")
```

```
	private int idAsignatura;

	@Column(name="nombre")
	private String nombre;
	@ManyToOne
	@JoinColumn(name="idprofesor")
	private Profesor profesor;

	public Asignatura() {}
	public Asignatura(String nombre, Profesor p) {
		this.nombre = nombre;
		profesor = p;}
	public int getIdAsignatura() {return idAsignatura;}
	public void setIdAsignatura(int idAsignatura) {
		this.idAsignatura = idAsignatura;}
	public String getNombre() {return nombre;}
	public void setNombre(String nombre) {this.nombre = nombre;}
	public Profesor getProfesor() {return profesor;}
	public void setProfesor(Profesor p) {profesor = p;}
}
```

Por último, ya podemos trabajar con dichas clases:

```
Profesor profesor=new Profesor("Pablo","pabloarellano@academiatheglobe.com");
List<Asignatura> as=new ArrayList<Asignatura>();
as.add(new Asignatura("Criptografia",profesor));
as.add(new Asignatura("Desarrollo web",profesor));
profesor.setListaAsignaturas(as);
// Solo queda persistir dichos objetos en la base de datos
```

Ejemplo: Clases con anotaciones relación muchos a muchos. Un profesor da clases en uno o varios cursos y en un curso puede dar clases uno o varios profesores. Se establece una relación muchos a muchos entre profesores y cursos.

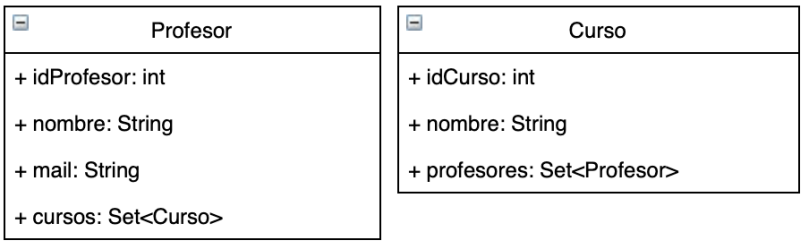
A nivel de base de datos tenemos 3 tablas:

```
CREATE TABLE profesores (
	idprofesor int(11) NOT NULL AUTO_INCREMENT,
	nombre varchar(100) NOT NULL,
	mail varchar(100),
	PRIMARY KEY (idprofesor) );

CREATE TABLE cursos (
	idcurso int(11) NOT NULL AUTO_INCREMENT,
	nombre varchar(100) NOT NULL,
	PRIMARY KEY (idcurso) );

CREATE TABLE profesorescursos (
	idprofesor int(11) NOT NULL,
	idcurso int(11) NOT NULL,
	PRIMARY KEY (idprofesor, idcurso) );
```

A nivel de clases tenemos:




```
import java.util.Set;
import jakarta.persistence.*;

@Entity
@Table(name="profesores")
public class Profesor implements Serializable {
	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	@Column(name="idprofesor")
	private int idProfesor;

	@Column(name="nombre")
	private String nombre;

	@Column(name="mail")
	private String mail;

	@ManyToMany(cascade="CascadeType.ALL)
	@JoinTable(name="profesorescursos",
			JoinColumns={@JoinColumn(name="idprofesor")},
			inverseJoinColumns={@JoinColumn(name="idcurso")})
	private Set<Curso> cursos;

	public Profesor() {}
	public Profesor(String nombre, String mail) {
		this.nombre = nombre;
		this.mail = mail;}
	public int getIdProfesor() { return idProfesor; }
	public void setIdProfesor(int idProfesor) {this.idProfesor = idProfesor;}
	public String getNombre() {return nombre;}
	public void setNombre(String nombre) {this.nombre = nombre;}
	public String getMail() {return mail;}
	public void setMail(String mail) {this.mail = mail;}
	public Set<Curso> getCursos() {return cursos;}
	public void setCursos(Set<Curso> c) {cursos = c;}
}

import java.util.Set;
import jakarta.persistence.*;
@Entity
@Table(name="cursos")
public class Curso implements Serializable {
	@Id
	@GeneratedValue(strategy=GenerationType.AUTO)
	@Column(name="idcurso")
	private int idCurso;

	@Column(name="nombre")
	private String nombre;
```

```
        @ManyToMany(mappedBy="cursos")
        private Set<Profesor> profesores;
        public Curso() {}
        public int getIdCurso() { return idCurso; }
        public void setIdCurso(int idCurso) {this.idCurso = idCurso;}
        public String getNombre() {return nombre;}
        public void setNombre(String nombre) {this.nombre = nombre;}
        public Set<Profesor> getProfesores() {return profesores;}
        public void setProfesores(Set<Profesor> p) {profesores = p;}
}
```

Por último, ya podemos trabajar con dichas clases:

```
Profesor p1=new Profesor("Pablo", "pabloarellano@academiatheglobe.com");
Profesor p2=new Profesor("Alex", "alejandro@academiatheglobe.com");
Curso c1=new Curso("Primero");
Curso c2=new Curso("Segundo");
Curso c3=new Curso("Tercero");
p1.getCursos().add(c1);
p1.getCursos().add(c2);
p2.getCursos().add(c3);
c1.getProfesores().add(p1);
c2.getProfesores().add(p1);
c3.getProfesores().add(p2);
// Solo queda persistir dichos objetos en la base de datos
```

Para completar el uso JPA nos queda ver cómo persistir de forma efectiva los objetos en la base de datos. Para ello utilizamos un **`EntityManager`**:

```
EntityManagerFactory emfactory = Persistence.createEntityManagerFactory("JPA");
EntityManager entitymanager = emfactory.createEntityManager( );
entitymanager.getTransaction( ).begin( );
//CREACIÓN DEL OBJETO A PERSISTIR
entitymanager.persist(OBJETO);
entitymanager.getTransaction().commit();
entitymanager.close();
emfactory.close();
```

Los métodos que oferta un `EntityManager` son:

- **persist()**: almacena el objeto en la base de datos.
- **find()**: búsqueda de un objeto en la base de datos.
- **remove()**: elimina el objeto de la base de datos.
- **flush()**: sincronización de los cambios contra la base de datos.

JPA proporciona los siguientes métodos para consultar entidades:

- El lenguaje de consulta Jakarta Persistence (**JPQL**) es un lenguaje simple basado en cadenas similar al SQL utilizado para consultar entidades y sus relaciones.
- La **API Criteria** se utiliza para crear consultas de tipo seguro utilizando las API de lenguaje de programación Java para consultar entidades y sus relaciones.

De otra parte, JPA también nos permite seguir el sentido inverso, creando objetos a partir de las tablas de una base de datos, y también de forma transparente.

De la misma forma que sucede en JDBC, para una clase de entidad tendremos que diseñar una **clase de servicio** para que persista los objetos de la clase entidad en la base de datos. Esta clase de servicio implementará el patrón DAO (Data Acces Object). De esta manera independizamos nuestra aplicación del framework concreto de persistencia que utilicemos. Es decir, si cambiamos de framework bastará adaptar las clases de servicio DAO sin que afecte al resto de nuestro sistema.

Ejemplo: clase de servicio `UserDAO` en Hibernate para persistir objetos de la clase `User`. El mapeo se hace con anotaciones en la clase `User` o mediante fichero XML. Aprovechamos el ejemplo para ver los métodos de consulta y persistencia y el uso del objeto `Session`.

```
import common.HibernateUtil;
import dominio.User;
import java.util.List;
import org.hibernate.HibernateException;
import org.hibernate.Session;

public class UserDAO {
    public boolean checkLogin(User user)  throws Exception {
        Session session = HibernateUtil.getSessionFactory().openSession();
        boolean acceso = false;

        try {
            session.beginTransaction();
            String query = "FROM User WHERE user='"
                           +user.getUsername()+"' AND password='"
                           +user.getPassword()+"'";

            List lista = session.createQuery(query).list();
            if(lista.size()>0) acceso=true;
            session.getTransaction().commit();

        } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback(); }
        }
        finally { session.close(); }

        return acceso;
    }

    public User datosUsuario(String username) {
        User usuario = new User();
        Session session =
             HibernateUtil.getSessionFactory().openSession();
        try {
            session.beginTransaction();
            String query = "FROM User WHERE user='"+username+"'";
            usuario = (User) session.createQuery(query).uniqueResult();
            session.getTransaction().commit();

        } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback();}
        }
        finally { session.close(); }
```

```
        return usuario;
    }

    public boolean existeUsuario(String username) {
        Session session = HibernateUtil.getSessionFactory().openSession();
        boolean acceso = false;

        try {
        session.beginTransaction();
        String query = "FROM User WHERE user='"+username+"'";
        List lista = session.createQuery(query).list();

        if(lista.size()>0) acceso=true;
        session.getTransaction().commit();

        } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback();}
        }
        finally { session.close(); }

        return acceso;
    }

    public List<User> list() {
       Session session = HibernateUtil.getSessionFactory().openSession();
       List lista = null;

       try {
           session.beginTransaction();
           String query = "FROM User";
           lista = session.createQuery(query).list();
           session.getTransaction().commit();

        } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback();}
        }
        finally { session.close(); }

       return lista;
    }

   public void add(User user){
        Session session = HibernateUtil.getSessionFactory().openSession();

        try {
            session.beginTransaction();
            session.saveOrUpdate(user);
            session.getTransaction().commit();

        } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback();
            }
        }
        finally { session.close(); }   }

    public void update(User user){
```

```
        Session session = HibernateUtil.getSessionFactory().openSession();

        try {
           session.beginTransaction();
           session.saveOrUpdate(user);
           session.getTransaction().commit();

        } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback();}
        }
        finally { session.close(); }
    }

    public void delete(User user){
       Session session = HibernateUtil.getSessionFactory().openSession();

       try {
           session.beginTransaction();
           session.delete(user);
           session.getTransaction().commit();

       } catch (HibernateException e) {
            if (session.getTransaction()!=null) {
                session.getTransaction().rollback();}
        }
        finally { session.close(); }
    }
}
```

```
public class HibernateUtil {
  private static SessionFactory sessionFactory;

  public static void buildSessionFactory() {
      if(sessionFactory == null) {
          Configuration conf = new Configuration().configure();
        StandardServiceRegistryBuilder builder =
            new StandardServiceRegistryBuilder().
            applySettings(conf.getProperties());
        sessionFactory = conf.buildSessionFactory(builder.build());
      }
  }

  public static SessionFactory getSessionFactory() {
      if(sessionFactory == null) {buildSessionFactory();}
      return sessionFactory;
  }
  public static void closeSessionFactory() {
      if((sessionFactory != null) && (!sessionFactory.isClosed())) {
          sessionFactory.close();}
  }
}
```

Por último, recordamos que JPA es una especificación de persistencia por lo que necesitamos de un framework de persistencia que implemente dicha especificación. Entre los **frameworks de persistencia** destacamos:

- **Hibernate**: ORM mediante archivos XML o anotaciones. Actualmente disponible en su versión 6, distribuido bajo licencia LGPL. Dispone de un lenguaje propio de consultas denominado **HQL** (Hibernate Query Language). Es el framework de persistencia por excelencia, el más utilizado. Tiene una variante, **Hibernate OGM**, que ofrece persistencia sobre bases de datos NoSQL.
- **EclipseLink** (de Eclipse Foundation).
- **Apache OpenJPA**.
- **Apache Cayenne**.
- **MyBatis** (antes iBATIS).
- **jOOQ**.
- **Ebean** (también para Kotlin).
- **Spring Data**.

JNDI

Para trabajar con directorios de nombres, se requiere el uso de clases e interfaces para acceder a dichos servicios de nombres, necesitándose el paquete:

```
import javax.naming.*;
```

6. Seguridad

Toda organización que tenga recursos confidenciales a los que puedan acceder muchos usuarios o recursos que atraviesen redes abiertas sin protección, como Internet, necesita protección.

La capa de negocio y web están formadas por componentes que se implementan en varios contenedores. Estos componentes se combinan para crear una aplicación empresarial de multicapa. La seguridad de los componentes es proporcionada por sus contenedores. Un contenedor proporciona 2 tipos de seguridad: declarativa y programática.

- La **seguridad declarativa** expresa los requisitos de seguridad de un componente de la aplicación mediante descriptores de implementación o anotaciones.
  Un descriptor de implementación es un archivo XML externo a la aplicación y que expresa la estructura de seguridad de una aplicación, incluidos los roles de seguridad, el control de acceso y los requisitos de autenticación.

  Las anotaciones, también llamadas metadatos, se utilizan para especificar información sobre seguridad dentro de un archivo de clase. Cuando se despliega la aplicación, el descriptor de despliegue de la aplicación puede usar o anular esta información. Las anotaciones le ahorran tener que escribir información declarativa dentro de los descriptores XML. En cambio, simplemente coloca anotaciones en el código y se genera la información requerida.

- La **seguridad programática** está integrada en una aplicación y se utiliza para tomar decisiones de seguridad. La seguridad programática es útil cuando la seguridad declarativa por sí sola no es suficiente para expresar el modelo de seguridad de una aplicación.

  La API para seguridad programática consta de métodos de la interfaz `Jakarta Security SecurityContext` y métodos de la interfaz Jakarta Enterprise Beans `EJBContext` y la interfaz del servlet `HttpServletRequest`. Los métodos de la interfaz Jakarta Security `SecurityContext` están destinados a reemplazar los métodos correspondientes de las interfaces `EJBContext` y `HttpServletRequest`.

Mecanismos de seguridad Java SE

Java SE proporciona soporte para una variedad de características y mecanismos de seguridad:

- **Java Cryptography Architecture** (JCA) contiene una arquitectura de proveedor y un conjunto de APIs para firmas digitales, hashes (SHA), validación de certificados y certificados, cifrado simétrico (DES, 3DES, IDEA…), cifrado asimétrico (RSA, DSA, ElGamal…), generación y gestión de claves y generación segura de números aleatorios. Reemplaza y amplia al API Java Cryptography Extension (JCE).

  Java ofrece para el manejo de la seguridad los siguientes paquetes principales: `java.security`, `javax.crypto`.

  Bouncy Castle es un proyecto de software libre que desarrolla una serie de APIs criptográficas libres y, entre otros, ofrece un proveedor (provider) para el JCA de Java. También, disponible para C#.

- **Java Authentication and Authorization Service** (JAAS) es un conjunto de APIs que permite que los servicios autentiquen y apliquen controles de acceso a los usuarios. JAAS proporciona un marco del módulo de autenticación conectable (PAM) para la autenticación y autorización de usuarios.

- **Java Generic Security Services** (Java GSS-API) es una API basada en token utilizada para intercambiar mensajes de forma segura entre aplicaciones que se comunican. El GSS-API ofrece acceso uniforme a servicios de seguridad además de una variedad de mecanismos de seguridad subyacentes, incluido Kerberos.

- **Java Secure Sockets Extension** (JSSE) proporciona un marco y una implementación para una versión Java de los protocolos TLS y DTLS (Datagram Transport Layer Security) e incluye funcionalidad para el cifrado de datos, la autenticación del servidor, la integridad del mensaje y la autenticación opcional del cliente para habilitar comunicaciones seguras de Internet.

- **Simple Authentication and Security Layer** (SASL) es un estándar de Internet (RFC 2222) que especifica un protocolo para la autenticación y el establecimiento opcional de una

capa de seguridad entre las aplicaciones cliente y servidor. SASL define cómo se intercambiarán los datos de autenticación, pero no especifica el contenido de esos datos. SASL es un marco en el que pueden encajar mecanismos de autenticación específicos que especifican el contenido y la semántica de los datos de autenticación.

Java SE también proporciona un conjunto de herramientas para administrar almacenes de claves, certificados y archivos de políticas, generar y verificar firmas jar y obtener, enumerar y administrar tickets de Kerberos.

Mecanismos de seguridad Jakarta EE

Los servicios de seguridad de Java EE son proporcionados por el contenedor de componentes y pueden implementarse utilizando técnicas declarativas o programáticas. Los servicios de seguridad de Java EE proporcionan un mecanismo de seguridad robusto y fácil de configurar para autenticar a los usuarios y autorizar el acceso a las funciones de la aplicación y los datos asociados en muchas capas diferentes. Los servicios de seguridad Java EE están separados de los mecanismos de seguridad del sistema operativo.

(1) Seguridad de la capa de aplicación

Los contenedores de componentes son responsables de proporcionar seguridad de la capa de aplicación, servicios de seguridad para un tipo de aplicación específico adaptado a las necesidades de la aplicación. En la capa de aplicación, los firewalls de la aplicación se pueden usar para mejorar la protección de la aplicación al proteger el flujo de comunicación y todos los recursos de la aplicación asociados contra ataques.

La seguridad de Jakarta EE es fácil de implementar y configurar y puede ofrecer un control de acceso preciso a las funciones y datos de la aplicación. Sin embargo, como es inherente a la seguridad aplicada en la capa de aplicación, las propiedades de seguridad no son transferibles a las aplicaciones que se ejecutan en otros entornos y protegen los datos solo mientras residen en el entorno de la aplicación. En el contexto de una aplicación empresarial tradicional, esto no es necesariamente un problema, pero cuando se aplica a una aplicación de servicios web, en la que los datos a menudo viajan a través de varios intermediarios, hay que usar los mecanismos de seguridad Java EE junto con la seguridad de la capa de transporte y seguridad de capa de mensaje para una solución de seguridad completa.

(2) Seguridad de la capa de transporte

Es proporcionada por los mecanismos de transporte utilizados para transmitir información. Se basa en HTTPS (TLS). La seguridad de transporte es un mecanismo de seguridad punto a punto que puede usarse para autenticación, integridad de mensajes y confidencialidad. Cuando se ejecuta sobre una sesión protegida por TLS, el servidor y el cliente pueden autenticarse entre sí y negociar un algoritmo de cifrado y claves criptográficas antes de que el protocolo de la aplicación transmita o reciba su primer byte de datos. La seguridad está activa desde el momento en que los datos salen del cliente hasta que llega a su destino, o viceversa, incluso a través de intermediarios. El problema es que los datos no están protegidos una vez que llegan al destino. Una solución es cifrar el mensaje antes de enviarlo.

(3) Seguridad de la capa de mensajes

En la seguridad de la capa de mensajes, la información de seguridad está contenida en el mensaje SOAP y/o el archivo adjunto del mensaje SOAP, lo que permite que la información de seguridad viaje junto con el mensaje o archivo adjunto. Cuando se envía desde el remitente inicial, el mensaje puede pasar a través de nodos intermedios antes de llegar a su receptor previsto. En este escenario, las partes encriptadas continúan siendo opacas para cualquier nodo intermedio y solo pueden ser desencriptadas por el receptor previsto. Por esta razón, la seguridad de la capa de mensajes también se conoce como seguridad de extremo a extremo.

Las tecnologías que nos ayudan a implementar la seguridad en una aplicación Jakarta EE son:

- **Jakarta Security**.
- **Jakarta Authentication**.
- **Jakarta Authorization**.

7. Empaquetamiento y despliegue de aplicaciones

Los componentes Jakarta EE deben ser empaquetados para facilitar su posterior instalación en el servidor Jakarta EE. La especificación Jakarta EE define cómo se debe realizar este empaquetado de modo que cualquier servidor compatible con las especificaciones pueda instalar el paquete.

Una aplicación Jakarta EE está compuesta, en principio, por componentes EJB, componentes web (servlets y/o JSP), clases con utilidades de servidor, componentes de cliente y ficheros HTML. Todos estos componentes se deben empaquetar junto con unos ficheros de configuración conocidos como descriptores de despliegue o deployment descriptors.

Los tipos de empaquetado son:

- **Jakarta/Java Archive** (**.jar**): los ficheros en formato JAR están basados en el formato ZIP y se utilizan para empaquetar un conjunto de ficheros. Los ficheros que conforman nuestra aplicación y los ficheros de metainformación que se pueden utilizar para la configuración de las clases incluidas forman parte del .jar.
- **Web Archive** (**.war**): este tipo de empaquetamiento permite la posibilidad de agrupar toda la estructura de directorios vista en .jar más el *deployment descriptor* (fichero de configuración web.xml) en un único fichero de extensión .war. El empaquetado de un .war es igual a un fichero .jar. Sin embargo, como diferencia un Web Archive puede ser desplegado directamente en el servidor de aplicaciones, ya que las aplicaciones webs no se instalan, se despliegan.
- **Enterprise Archive** (**.ear**): empaquetamiento asociado básicamente a los EJB, aunque también se puede utilizar en aplicaciones web. Un fichero EAR puede contener uno o varios ficheros WAR, uno o varios ficheros JAR los cuales albergan los EJB de nuestra aplicación, uno varios ficheros JAR con las clases de servicio o utilidad y un *deployment descriptor*.

Otros formatos basados en .jar son:

- **SAR (Service Archive)**: similar a EAR; proporciona un fichero service.xml y los archivos JAR correspondientes. Usado en SOA.
- **RAR (Resource Adapter Archive)**: almacena archivos XML, clases de Java y otros objetos para aplicaciones de Java EE Connector Architecture (JCA). Un archivo RAR contiene un archivo JAR y opcionalmente un fichero ra.xml.

8. Frameworks y otras APIs

Frameworks

- **Apache Axis2**: framework para desarrollo de servicios web SOAP y RESTful.
- **Apache CXF**: framework que soporta servicios web SOAP y RESTful.
- **Apache Wicket**: framework de desarrollo de aplicaciones web.
- **Jersey**: framework de Eclipse con soporte para servicios web RESTful en Java.
- **Google Web Toolkit** (**GWT**): framework de código abierto desarrollado por Google para crear y optimizar aplicaciones web de alto rendimiento basadas en navegador (JS, AJAX a través de XMLHttpRequest) sin que el desarrollador tenga que ser un experto.
- **Grails**: framework escrito en Groovy orientada a crear aplicaciones web de forma sencilla que adopta el principio DRY.
- **Guice**: framework de inyección de dependencias utilizado en aplicaciones Java creado por Google.
- **JHipster**: framework de código abierto que se utiliza para desarrollar aplicaciones web y microservicios.
- **Jena**: framework de Apache para construir aplicaciones de Web Semántica y Datos Vinculados en Java.
- **Micronaut**: framework JVM que permite crear aplicaciones basadas en microservicios de una manera sencilla y rápida. Además de Java, Micronaut permite el uso de otros lenguajes como Groovy y Kotlin.
- **MINA**: framework de Apache para desarrollar aplicaciones de red de alto rendimiento y alta escalabilidad.
- **Play**: framework de desarrollo web de código abierto.

- **PrimeFaces**: framework de código abierto con un conjunto de componentes para JSF.

- **RESTEasy**: framework para desarrollo de servicios web RESTful en Java (JAX-RS).

- **Spring**: framework que agiliza el desarrollo de aplicaciones empresariales. Este framework de código abierto implementa un contenedor de inversión de control (IoC), puede ser usado para la programación aplicaciones web o de escritorio estándar y cuenta con gran variedad de módulos que nos facilitan el trabajo.

- **Spring Boot**: subproyecto de Spring MVC que permite el desarrollo de aplicaciones web autocontentidas que llevan embebido el contenedor de servlets y también de aplicaciones basadas en microservicios.

- **Spring MVC**: subproyecto de Spring dirigido a facilitar y optimizar el proceso de creación de aplicaciones web utilizando el patrón MVC (Modelo Vista Controlador), donde el Modelo representa los datos o información que manejará la aplicación web, la Vista son todos los elementos de la UI (Interfaz de Usuario), con los que el usuario final interactúa con la aplicación y el Controlador es el encargado de manipular los datos en según la interacción del usuario.

- **Spring Web Services** (Spring-WS): facilita el desarrollo de servicios web SOAP.

- **Struts 2**: framework de código abierto de Apache para el desarrollo de aplicaciones web siguiendo el patrón de diseño MVC. Permite la integración con Spring e Hibernate.

- **VRaptor**: framework MVC de código abierto.

Librerías y herramientas

- **Ant**: herramienta de Apache para compilación y construcción.

- **Commons**: conjunto de componentes Java reutilizables (de Apache, https://commons.apache.org/).

- **DBUnit**: extensión de JUnit para realizar pruebas del funcionamiento de las operaciones de base de datos.

- **Google Guava**: conjunto de bibliotecas comunes Java de código abierto.

- **HttpComponents**: conjunto de componentes Java de bajo nivel centrados en HTTP (de Apache), como HttpCore o HttpClient.

- **iText**: generación de archivos en formato PDF.

- **Ivy**: administrador de dependencias de Apache, parte de Ant.

- **Jackson**: procesador JSON (de Apache).

- **JasperReports**: permite la generación de informes. Bibliotecas alternativas: Birt Reports y FineReport.

- **Javadoc**: herramienta para generar documentación API en formato HTML a partir de comentarios de documentos en código fuente.

- **JMeter**: herramienta de Apache para realizar pruebas de estrés o rendimiento en Java.

- **JUnit**: biblioteca que evalúa el funcionamiento de nuestro desarrollo mediante pruebas funcionales de clases y métodos. Última versión estable, 5.

- **Maven**: herramienta de Apache para gestión y construcción de proyectos Java. La configuración del proyecto a través de ficheros XML (pom.xml). Permite la gestión de dependencias clara y sencilla.

- **Mockito**: herramienta de Apache para realizar pruebas unitarias, basado en mock (objetos simulados).

- **log4j**: implementa un sistema de log (de Apache).

- **Lucene**: biblioteca de Apache que implementa un motor de búsqueda de texto para recuperación de información.

- **SLF4J** (Simple Logging Facade for Java): ofrece una fachada para varios frameworks de loggin.

- **Spring Cloud**: proporciona herramientas para implementar patrones en sistemas distribuidos como descubrimiento de servicios, circuit breakers, enrutamiento… Usado en Spring en el desarrollo de aplicaciones basadas en microservicios.

- **Spring Data**: encargado del acceso a los datos. Uso asociado al framework Spring.

- **Spring Security**: gestiona la seguridad de la aplicación. Uso asociado al framework Spring o también de forma independiente.

- **Thymeleaf**: motor de plantillas Java del lado del servidor que permite la creación de páginas web con representación. Tiene un uso frecuente en combinación con el framework Spring.



- **Velocity**: proyecto de Apache que es un motor de plantillas basado en Java.

2. PLATAFORMA .NET

.NET es una plataforma de desarrollo de uso general, multiplataforma (Android, Apple, Linux y Windows) y de código abierto (licencia MIT) para crear tipos diferentes de aplicaciones.

Tiene varias características clave, como la compatibilidad con varios lenguajes de programación, modelos de programación asincrónica y simultánea e interoperabilidad nativa, que permiten una amplia variedad de escenarios en diversas plataformas. C# es el lenguaje de programación de referencia para .NET.

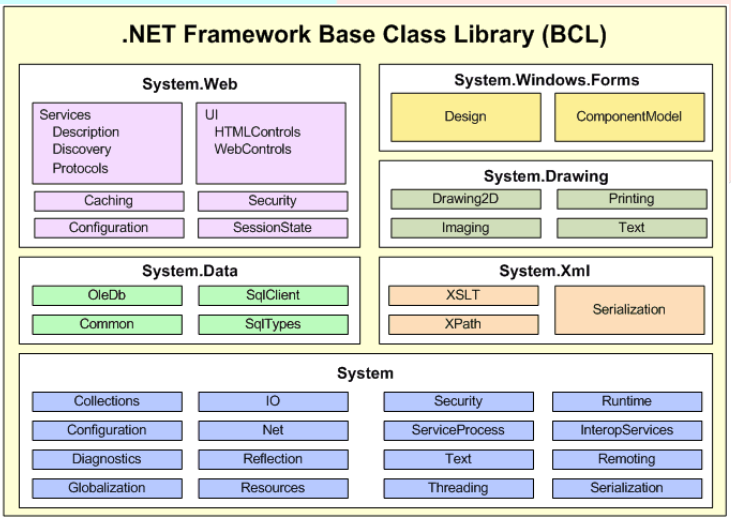
.NET Standard es una **especificación** formal de las API (biblioteca) de .NET que son **comunes** en todas las implementaciones de .NET (.NET Framework, .NET Core, Mono, Xamarin, UWP y Unity).

.NET Standard permite que las bibliotecas se compilen con el conjunto acordado de API comunes, lo que garantiza que se puedan usar en cualquier aplicación .NET: móvil, escritorio, IoT, web…

La última versión de .NET Standard es la 2.1 y Microsoft no publicará ninguna versión más. .NET y .NET 5 y versiones posteriores se convierten en la mejor forma de compartir código adoptando un enfoque diferente para establecer la uniformidad que elimine la necesidad de .NET Standard. El motivo de abandonar .NET Standard en favor de .NET es no separar la especificación de la implementación de las API y, también, la ampliación rápida de nuevas funcionalidades a la API evitando procesos previos de revisión de propuestas.

.NET Standard ofrece las siguientes ventajas:

- Define un conjunto uniforme de API denominada **BCL** (Biblioteca de Clases Base) para todas las implementaciones de .NET. La BCL conforma utiliza el espacio de nombres de **`System.*`**.



- Permite a los desarrolladores generar bibliotecas portables que se pueden usar en las implementaciones de .NET con este mismo conjunto de API.

Las implementaciones actuales de .NET (4) son:

- **.NET Framework**: entorno clásico. El runtime utilizado es CLR (Common Language Runtime). Se suele utilizar el término .NET para hace referencia a esta implementación ya que es la implementación de .NET original. Ha empezado a cumplir .NET Standard desde la versión 4.5.

NEW

- **.NET 10 (antes .NET Core)**: de código abierto, multiOS (Windows, macOS y Linux). El runtime utilizado es coreCLR.

  Microsoft en los últimos años ha hecho un esfuerzo para otorgar más protagonismo a su comunidad de desarrollo y al software libre. Fruto de ello es la creación de .NET Core (actual .NET 10), implementación de .NET Standard, pero orientada desde el principio a la portabilidad entre plataformas (Windows, Linux y MacOS, y también con Docker), sin dependencias específicas de Windows.

- **Mono**: de código abierto y multiplataforma, tiene su origen en Linux. El runtime utilizado es Mono runtime. Se usa principalmente cuando se requiere un entorno de ejecución pequeño.

- **UWP (Universal Windows Platform)**: propio de Windows, usado para compilar aplicaciones Windows modernas y táctiles y software para IoT. El runtime utilizado es .NET Native para UWP. Busca ofrecer a desarrolladores un entorno único de desarrollo que permite generar aplicaciones para las diferentes versiones de Windows (tabletas, PCs, Xbox, etc.).

Desde mayo de 2024, Microsoft ha dejado de prestar soporte para la implementación **Xamarin**, ya que se ha integrado en .NET. Es de código abierto y multiplataforma, y su fin era crear aplicaciones Android, iOS, tvOS, macOS y Windows.



Por último, destacamos **UNITY**, una plataforma de desarrollo 3D en tiempo real para compilar aplicaciones 2D y 3D, como juegos y simulaciones, con .NET y el lenguaje de programación C#.

Cada implementación tiene su propio runtime: CLR es el clásico para .NET Framework, CoreCLR es el de .NET 9, Mono runtime el de Mono y .Net Native para UWP.

Los lenguajes de programación que ofrece .NET son:

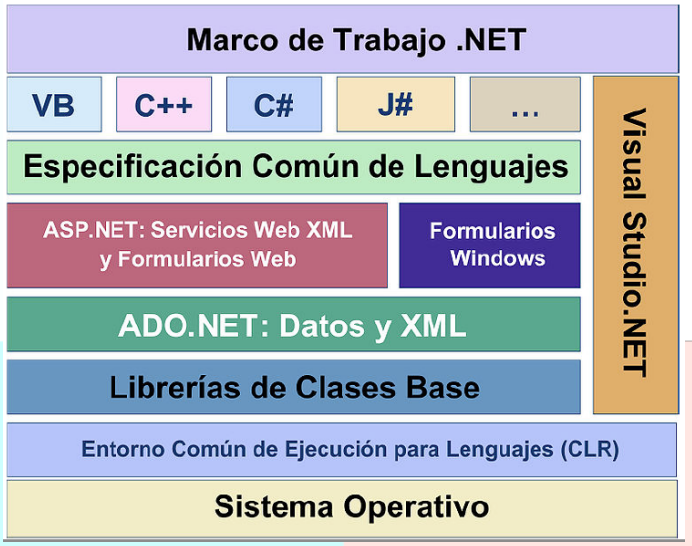
- C#.
- F#.
- Visual Basic.

Un lenguaje de programación que permita desarrollar aplicaciones en .NET Framework debe cumplir las especificaciones del **Common Language Infrastructure** (**CLI**), definido en el estándar ECMA-335 y en ISO/IEC 23271.

1. .NET Framework

.NET Framework es el núcleo de la plataforma y ofrece la infraestructura necesaria para desarrollar y ejecutar aplicaciones .NET. Consta de 2 componentes principales:

- **Common Language Runtime (CLR)**: es el **motor de ejecución** que controla las aplicaciones en ejecución.
- **Biblioteca de clases de .NET Framework**: proporciona una **biblioteca** de clases reutilizable al que pueden llamar los desarrolladores desde sus propias aplicaciones.



Los **servicios** que ofrece .NET Framework a las aplicaciones en ejecución son los siguientes:

- **Administración de la memoria**. En muchos lenguajes de programación, los programadores son responsables de asignar y liberar memoria y de administrar la vida útil de los objetos. En las aplicaciones de .NET Framework, CLR proporciona estos servicios en nombre de la aplicación.

- **Sistema de tipos comunes (CTS, Common Type System)**. En los lenguajes de programación tradicionales, el compilador define los tipos básicos, lo que complica la interoperabilidad entre lenguajes. En .NET Framework, **los tipos básicos los define el CTS** de .NET Framework (por ejemplo, Boolean, Byte, Char…) son comunes a todos los lenguajes que tienen como destino .NET Framework. Esto permite la interoperabilidad entre los diferentes lenguajes de programación.

- **Biblioteca de clases extensa**.

- **Marcos de desarrollo y tecnologías** (ver apartado Biblioteca de clases).

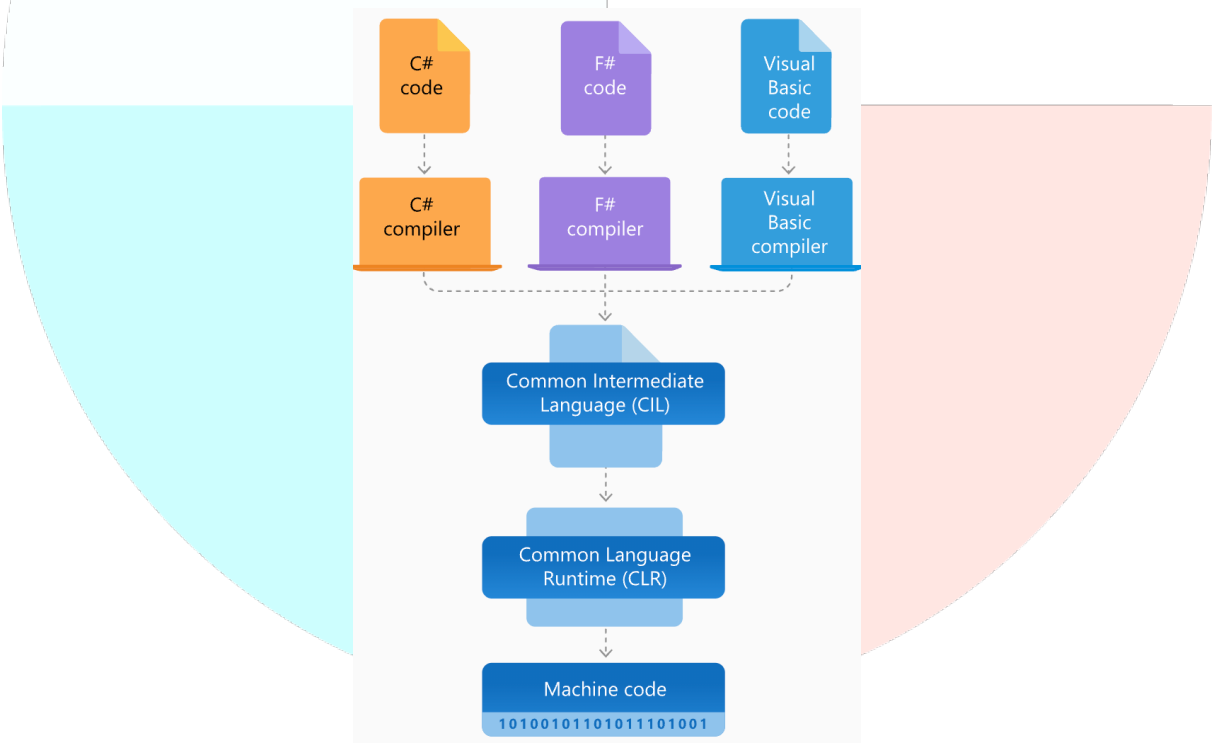
- **Interoperabilidad de lenguajes**. Los compiladores de lenguajes cuya plataforma de destino es .NET Framework emiten un código intermedio denominado Lenguaje intermedio común (**CIL**), que, a su vez, se compila en tiempo de ejecución a través del CLR. Con esta característica, el código escrito en un lenguaje de programación concreto es accesible desde otras aplicaciones escritas lenguajes distintos, de modo que los programadores puedan centrarse en crear aplicaciones en su lenguaje preferido.

- **Compatibilidad de versiones**. Como norma general, las aplicaciones que se desarrollan con una versión determinada de .NET Framework se ejecutan sin modificaciones en una versión posterior.

- **Ejecución en paralelo**. .NET Framework ayuda a resolver conflictos entre versiones y permite que varias versiones del CLR coexistan en el mismo equipo. Esto significa que pueden coexistir varias versiones de las aplicaciones, y que una aplicación se puede ejecutar en la versión de .NET Framework con la que se compiló.

- **Compatibilidad con múltiples versiones (multi-targeting)**. Al establecer .NET Standard como destino, los desarrolladores pueden crear bibliotecas de clases que funcionan en varias plataformas de .NET Framework compatibles con esa versión del estándar.

La arquitectura de .NET Framework se puede observar en la siguiente figura:



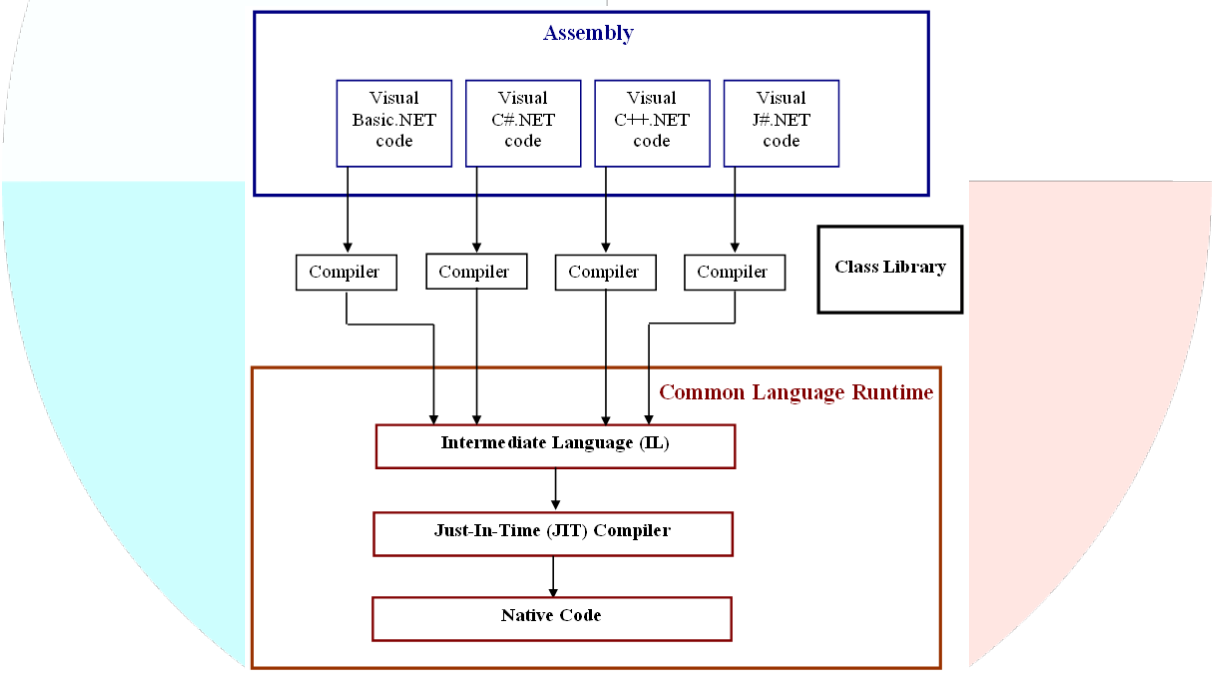
CLR

Common Language Runtime es el entorno donde se **ejecutan** todas las aplicaciones .NET.

La herramienta **Clrver.exe** notifica todas las versiones instaladas de CLR en el equipo. Esta herramienta se instala automáticamente con Visual Studio.

La herramienta de desarrollo compila el código fuente de cualquiera de los lenguajes soportados por .NET en un código intermedio o administrado denominado **Common Intermediate Language** (**CIL**, antes MSIL o IL, Microsoft Intermediate Lenguaje), similar al bytecode de Java. Para generar dicho código el compilador se basa en el **Common Language Specification** (**CLS**) que determina las reglas necesarias para crear ese código CIL compatible con el CLR.

Para ejecutarse se necesita un segundo paso, un compilador **JIT** (Just-In-Time) es el que genera el código máquina real que se ejecuta en la plataforma del cliente y se lleva a cabo en el mismo equipo en que debe ejecutarse el código. Se requieren tantas compilaciones JIT como instancias de la aplicación se estén ejecutando. Puesto que la compilación JIT tiene lugar durante la ejecución de la aplicación, el tiempo de compilación es parte del tiempo de ejecución. Por tanto, los compiladores JIT tienen que compensar el tiempo invertido en optimizar el código con el ahorro que puede generar el código resultante. Pero un JIT conoce el hardware real y puede liberar a los desarrolladores de tener que enviar diferentes implementaciones.



AOT (Ahead Of Time) es un compilador similar a JIT, que convierte IL en código de máquina. A diferencia de la compilación JIT, la compilación AOT ocurre antes de que la aplicación se ejecute y, normalmente, se realiza en un equipo diferente.

De esta forma se consigue con .NET independencia de la plataforma hardware.

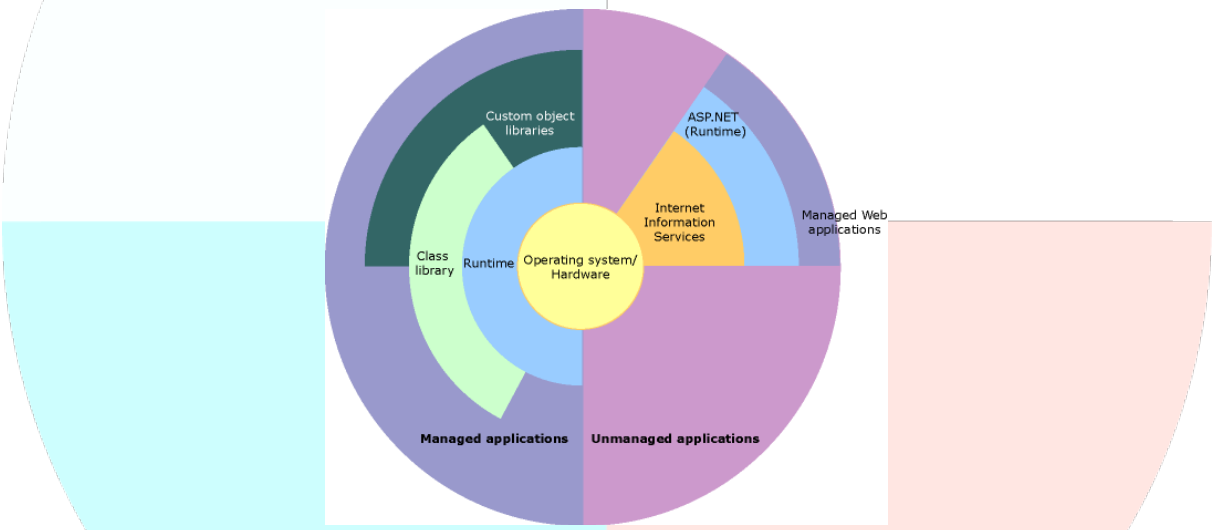
Servicios de CLR:

- Compilador de CIL a nativo.

- Recolector de basura (GC, Garbage Collector): es un recolector de elementos no utilizados que implementa la administración de memoria automática. GC libera la memoria ocupada por objetos que ya no se usan.
- Motor de seguridad.
- Motor de depuración.
- Verificador de tipos.
- Administrador de excepciones.
- Soporte multiproceso (threads).

CLR admite un modelo de seguridad denominado seguridad de acceso del código o **CAS** (Code Access Security) para el código administrado. En este modelo se conceden permisos a los ensamblados basados en la identidad del código.

El código puede ser administrado o no administrado. Si es administrado, la gestión de memoria, punteros… la realiza el CLR. Si no es administrado el CLR no participa en la ejecución del código, es el sistema operativo el que se encarga de ello.



Con respecto a la seguridad, los componentes administrados reciben grados de confianza diferentes en función de una serie de factores, entre los que se incluye su origen (como Internet, red empresarial o equipo local). Esto significa que un componente administrado puede ser capaz o no de realizar operaciones de acceso a archivos, operaciones de acceso al Registro y otras funciones delicadas, incluso si se usa en la misma aplicación activa.

El runtime impone la solidez del código mediante la implementación de una infraestructura estricta de comprobación de tipos y código (**CTS**, Common Type System) que garantiza que todo el código administrado sea autodescriptivo.

Ensamblados

Los **ensamblados** son las unidades de creación de las aplicaciones .NET Framework (concepto específico de este framework). Son la unidad fundamental de implementación, control de versiones, reutilización, ámbitos de activación y permisos de seguridad.

Un ensamblado es una **colección de tipos y recursos** compilados para funcionar en conjunto y formar una unidad lógica de funcionalidad (componente). Proporcionan al CLR la información necesaria para conocer las implementaciones de tipos.

Los ensamblados adoptan la forma de un archivo ejecutable .exe o de biblioteca .dll.

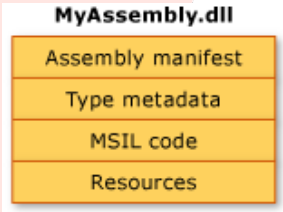
Cada ensamblado .exe se ejecuta en su propio dominio de aplicación (AppDomain), es decir, en su propio proceso de Windows. Sin embargo, los procesos host de CLR, como IIS (Internet Information Services) o Microsoft SQL Server, pueden decidir ejecutar varios dominios de aplicación en el mismo proceso de Windows.

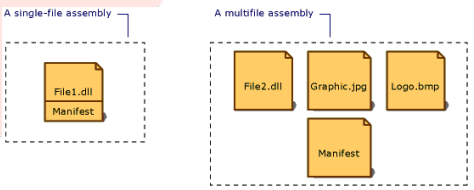
Un ensamblado ofrece (funciones):

- El **código** que ejecuta el CLR.
- Un **límite de seguridad**. Un ensamblado es la unidad en la que se solicitan y conceden los permisos.
- Un **límite de tipos**. La identidad de cada tipo incluye el nombre del ensamblado en que reside.
- Un **límite del ámbito de referencia**. El manifiesto del ensamblado contiene los metadatos que se usan para resolver tipos y satisfacer solicitudes de recursos.
- Un **límite de versión**. El ensamblado es la unidad versionable más pequeña de CLR. Todos los tipos y recursos del mismo ensamblado tienen la misma versión, como una unidad.
- Una **unidad de implementación**. Cuando se inicia una aplicación sólo deben estar presentes los ensamblados a los que llama la aplicación inicialmente.

Elementos de un ensamblado:

- **Manifiesto** del ensamblado, que contiene los metadatos del ensamblado (es el único elemento obligatorio):
  - Nombre ensamblado.
  - Referencia cultural: información sobre la referencia cultural o idioma que admite el ensamblado.
  - Nº versión.
  - Lista de archivos del ensamblado: código hash de cada archivo que contiene el ensamblado y nombre de archivo.
  - Información sobre otros ensamblados a los que hace referencia.
- **Metadatos** de tipos.
- **Código CIL**, el sign code permite firmar el ejecutable para garantizar la integridad.
- Conjunto de **recursos**.





Biblioteca de clases de .NET Framework

La biblioteca de clases de .NET Framework es una colección de clases reutilizables que se integran con CLR. La biblioteca de clases está orientada a objetos, lo que permite mediante herencia poder ampliar el código administrado. Esto hace que las clases de .NET Framework sean fáciles de usar y reduce el tiempo asociado con el aprendizaje de las nuevas características de .NET Framework. Además, los componentes de terceros se integran sin dificultades con las clases de .NET Framework.

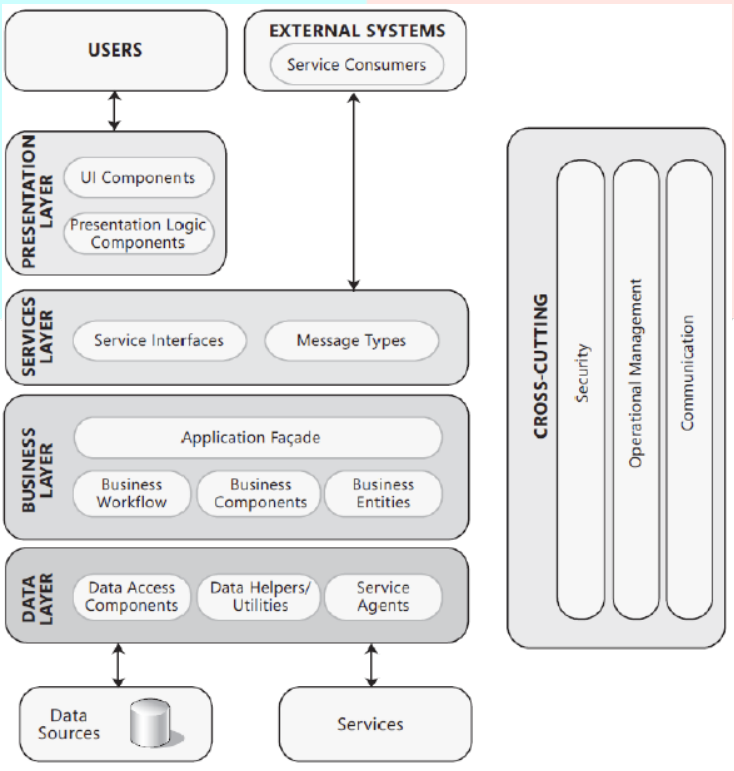
Como en cualquier biblioteca de clases orientada a objetos, las clases de .NET Framework permiten realizar diversas tareas de programación comunes, como son la administración de cadenas, la recolección de datos, la conectividad de bases de datos y el acceso a archivos.

Además de estas tareas habituales, la biblioteca de clases incluye tipos adecuados para diversos escenarios de desarrollo especializados, como:

- **ASP.NET** para aplicaciones web.
- **ADO.NET** para el acceso a los datos.
- **Windows Communication Foundation** (**WCF**) para las aplicaciones orientadas a servicios.
- **Windows Presentation Foundation** (**WPF**) para las aplicaciones de escritorio de Windows. **Windows Forms** como alternativa.
- **Windows Workflow Foundation (WF)**: para flujos de trabajo.

2. Arquitectura

A continuación, se muestra la arquitectura detallada de una aplicación en .NET:



Aplicación cliente

Podemos crear aplicaciones basadas en Windows mediante:

- **Windows Presentation Foundation** (WPF): permite crear aplicaciones de cliente de escritorio para Windows. El núcleo de WPF es un motor de representación basado en vectores e independiente de la resolución que está diseñado para sacar partido al moderno hardware gráfico. WPF amplía el núcleo con un conjunto completo de características de desarrollo de aplicaciones que incluyen **XAML** (Extensible Application Markup Language), controles, enlace de datos, diseño, gráficos en 2D y 3D, animación, estilos, plantillas, documentos, elementos multimedia, texto y tipografía. WPF es parte de. NET, así que permite compilar aplicaciones que incorporan otros elementos de la API de .NET.

  Ejemplo:

```
<Window
 xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    Title="Window with Button"
    Width="250" Height="100">

  <!-- Add button to window -->
  <Button Name="button">Click Me!</Button>
</Window>
```



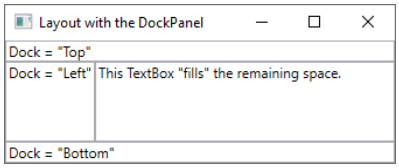
  La lista de controles de WPF son:

  - Botones: `Button` y `RepeatButton`.
  - Presentación de datos: `DataGrid`, `ListView` y `TreeView`.
  - Presentación y selección de fechas: `Calendar` y `DatePicker`.
  - Cuadros de diálogo: `OpenFileDialog`, `PrintDialog` y `SaveFileDialog`.
  - Entrada de lápiz digital: `InkCanvas` y `InkPresenter`.
  - Documentos: `DocumentViewer`, `FlowDocumentPageViewer`, `FlowDocumentReader`, `FlowDocumentScrollViewer` y `StickyNoteControl`.
  - Entrada: `TextBox`, `RichTextBox` y `PasswordBox`.
  - Diseño: `Border`, `BulletDecorator`, `Canvas`, `DockPanel`, `Expander`, `Grid`, `GridView`, `GridSplitter`, `GroupBox`, `Panel`, `ResizeGrip`, `Separator`, `ScrollBar`, `ScrollViewer`, `StackPanel`, `Thumb`, `Viewbox`, `VirtualizingStackPanel`, `Windowy WrapPanel`.
  - Multimedia: `Image`, `MediaElement` y `SoundPlayerAction`.
  - Menús: `ContextMenu`, `Menu` y `ToolBar`.
  - Navegación: `Frame`, `Hyperlink`, `Page`, `NavigationWindow` y `TabControl`.
  - Selección: `CheckBox`, `ComboBox`, `ListBox`, `RadioButton` y `Slider`.
  - Información del usuario: `AccessText`, `Label`, `Popup`, `ProgressBar`, `StatusBar`, `TextBlock` y `ToolTip`.

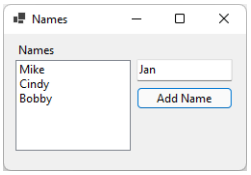
  Los controles se sitúan en contenedores (`Canvas`, `DockPanel`, `StackPanel`, `Grid`).

Ejemplo:

```
<Window xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    x:Class="SDKSample.LayoutWindow"
    Title="Layout with the DockPanel" Height="143" Width="319">
  <DockPanel> <!--DockPanel to layout four text boxes-->
    <TextBox DockPanel.Dock="Top">Dock = "Top"</TextBox>
    <TextBox DockPanel.Dock="Bottom">Dock = "Bottom"</TextBox>
    <TextBox DockPanel.Dock="Left">Dock = "Left"</TextBox>
    <TextBox Background="White">
      This TextBox "fills" the remaining space.
    </TextBox>
  </DockPanel>
</Window>
```



- **Windows Forms**: permite crear aplicaciones de escritorio tradicionales con componentes básicos IU. Se utiliza un formulario, como contenedor, en el que se pueden incluir controles `Label`, `Button`, `Listbox` y `TextBox`. Una vez definida la interfaz se implementan los manejadores de eventos sobre los controles.

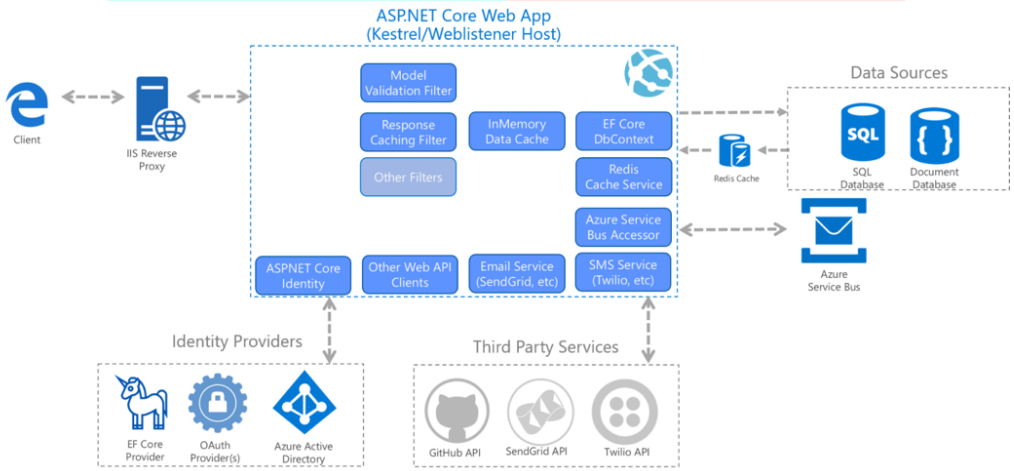


- **Silverlight**: permite la presentación de IU en navegadores web. Microsoft Chrome no soporta esta tecnología. El soporte técnico para Silverlight finalizó en 2021.

ASP.NET (Capa de presentación)

ASP.NET (Active Server Pages) es un conjunto de tecnologías del framework .NET para construir servicios y aplicaciones web en Windows. En ASP.NET "clásico" tanto aplicaciones como servicios web se despliegan en el servidor de aplicaciones IIS (Internet Information Services). El ensamblado `System.Web` contiene todas las clases de ASP.NET.

Sin embargo, con la llegada de Core, se definió un framework llamado "**ASP.NET Core**", extremadamente similar a ASP.NET clásico, pero multiplataforma y de código abierto con la finalidad de compilar aplicaciones tanto en local como destinadas a la nube, y que formó parte de .NET Core. De esta manera se pueden desarrollar aplicaciones de ASP.NET que sean portable entre los diferentes entornos que soporta .NET Core. Para el desarrollo de nuevas aplicaciones se debe usar ASP.NET Core.



Modelos/marcos de programación

El hecho de elegir uno de los modelos de programación al comenzar un proyecto de ASP.NET no excluye necesariamente a los otros, sino que es posible tener aplicaciones "híbridas" y en muchos casos tendrá todo el sentido desarrollar ciertas partes de la aplicación con un modelo de programación y otras partes con otro modelo distinto.

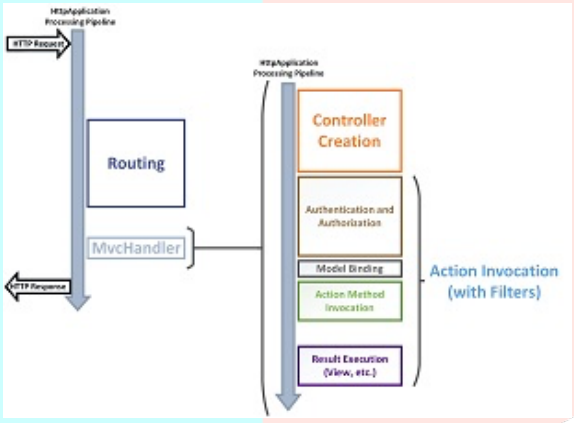
- **ASP.NET** Web Forms fue el primero de los modelos de programación en existir proporcionando un gran nivel de abstracción. Está basado en eventos y controles que favorecen la productividad mediante la programación declarativa, reduciendo la cantidad de código necesaria para implementar una determinada funcionalidad. El código se ejecuta en el servidor y genera dinámicamente la salida de la página web (HTML) al navegador o al dispositivo cliente.

- **ASP.NET** MVC se concibió como alternativa a Web Forms y proporciona un modelo de programación basado en el patrón de diseño MVC. Entre sus principales características destacan su completa integración con pruebas unitarias y la separación más clara entre la lógica de presentación (vista), la lógica de negocio (controlador) y la lógica de datos (datos).

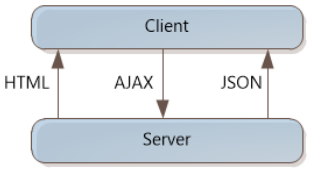
  Pueden existir más de un controlador en una app. La sintaxis para el nombre de un controlador es `nombre`***`Controller`***.

  ASP.NET MVC invoca a un controlador concreto y dentro de él a un método de acción específico en función de la dirección URL entrante, que sigue la siguiente sintaxis:

  ```
  /[Controller]/[ActionName]/[Parameters]
  ```



  ASP.NET MVC incluye una plantilla de aplicación de página única (SPA, Single Page Application) para crear una aplicación web que carga una sola página HTML y después la actualiza dinámicamente, en lugar de cargar páginas nuevas. Tras la carga inicial de la página, la SPA habla con el servidor a través de solicitudes AJAX.



- **ASP.NET** Web Pages es el más reciente de los modelos de programación, perteneciente a .NET Core. Utiliza la sintaxis **Razor**. Fue creado como respuesta a una creciente demanda

de desarrolladores web sin experiencia previa con ASP.NET, cuya iniciación en ASP.NET Web Forms o MVC les suponía una inversión inicial de tiempo demasiado grande. Web Pages proporciona un modelo de programación más simple y rápido de aprender, sin renunciar a toda la funcionalidad y flexibilidad de ASP.NET. Cada página de Razor se compone de un par de archivos:

- .cshtml: contiene el marcado HTML con código C# que usa la sintaxis Razor.
- .cshtml.cs: contiene código C# que controla los eventos de página.

Conceptos básicos sobre ASP.NET Web Pages:

- El carácter `@` indica a ASP.NET que lo que sigue es código Razor, no HTML. ASP.NET tratará todo lo que haya después del carácter `@` como código hasta que vuelva a ejecutarse en HTML (ejemplo `@{código}`). También `@` es utilizado en expresiones. La sintaxis es `@expresion`.

- Las llaves `{}` encierran un bloque de código Razor si el código tiene más de una línea. Las llaves indican a ASP.NET dónde se inicia y finaliza el código para ese bloque.

- Los caracteres `//` marcan un comentario, es decir, una parte del código que no se ejecutará.

- Cada instrucción tiene que terminar con un punto y coma `;`.

- Puede almacenar valores en variables, que declare con la palabra reservada `var`. Cuando se crea una variable, se le asigna un nombre, que puede incluir letras, números y subrayados. Los nombres de variable no pueden comenzar con un número y no pueden usar el nombre de una palabra reservada del lenguaje.

- Las cadenas de caracteres (como "ASP.NET" y "páginas web") se delimitan entre comillas dobles.

- Expresiones Razor implícitas: comienzan por `@`, seguido de código C#.

  Ejemplo: `<p>@DateTime.Now</p>`

- Expresiones Razor explícitas: constan del símbolo `@` y un paréntesis de apertura y de cierre. El contenido situado entre paréntesis `@()` se representa en la salida.

  Ejemplo:

  ```
  <p>Last week this time: @(DateTime.Now - TimeSpan.FromDays(7))</p>
  ```

- Las estructuras condicionales permitidas son `@if`, `else if`, `else` y `@switch`. Notar que `else if` y `else` no necesitan `@`.

- Las estructuras repetitivas soportadas son `@for`, `@foreach`, `@while`, y `@do while`.

  Ejemplo:

  ```
  @foreach (var person in people){
      <p>Name: @person.Name</p>
  ```

```
        <p>Age: @person.Age</p>
    }
```

Ejemplo:

```
@{
    // Working with numbers
    var a = 4;
    var b = 5;
    var theSum = a + b;

    // Working with characters (strings)
    var technology = "ASP.NET";
    var product ="Web Pages";

    // Working with objects
    var rightNow = DateTime.Now;
}

<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Testing Razor Syntax</title><meta charset="utf-8" />
  </head>
<body>
  <h1>Testing Razor Syntax</h1>
  <form method="post">
    <div>
      <p>The value of <em>a</em> is @a.  The value of <em>b</em> is @b.</p>
      <p>The sum of <em>a</em> and <em>b</em> is
            <strong>@theSum</strong>.
      </p>
      <p>The product of <em>a</em> and <em>b</em> is
            <strong>@(a*b)</strong>.
      </p>
    </div>

    <div>
      <p>The technology is @technology, and the product is @product.</p>
      <p>Together they are
            <span class="bright">@(technology + " " + product)</span>
      </p>
    </div>

    <div>
      <p>The current date and time is: @rightNow</p>
      <p>The URL of the current page is<br/><br/>
            <code>@Request.Url</code>
      </p>
    </div>
  </form>
</body>
</html>
```

The value of *a* is 4. The value of *b* is 5.

The sum of *a* and *b* is **9**.

The product of *a* and *b* is **20**.

Las **directivas** se representan con la palabra reservada en cuestión antecedido por el símbolo @. Vamos a nombrar las directivas más destacables:

- @page: indica que el archivo es una página Razor.

- `@using`: uso de espacios de nombres de la BCL.
- `@namespace`: establece el espacio de nombre de la clase de la página Razor.
- `@layout`: especifica un diseño para un componente Razor.
- `@implements`: implementa una interfaz para la clase generada.

Respecto al manejo de **sesiones**, podemos configurar el estado de una sesión. Para ello, se utiliza el paquete `Microsoft.AspNetCore.Session`. Se puede configurar la creación de cookies o el tiempo de inactividad de una sesión antes de ser cerrada.

Finalmente, destacamos el objeto **`Request`**. Contiene toda la información que el explorador envía a la aplicación cuando se solicita o envía una página. Este objeto incluye cualquier información que el usuario proporcione, como valores de campos de texto o un archivo para cargar. También incluye todo tipo de información adicional, como cookies, valores en la cadena de consulta URL (si existe), la ruta de acceso del archivo de la página que se está ejecutando, el tipo de navegador que utiliza el usuario, la lista de idiomas que se establecen en el explorador y mucho más.

El objeto `Request` es una colección (lista) de valores. Para obtener un valor individual de la colección, especificamos su nombre:

```
var someValue = Request["name"];
```

El Request objeto expone realmente varios subconjuntos, entre ellos:

- **`Request.Form`**: proporciona valores de elementos `<form>` dentro del elemento POST enviado si la solicitud es una solicitud.
- **`Request.QueryString`**: le da solo los valores de la cadena de consulta de la dirección URL. Por ejemplo, en `http://mysite/myapp/page?searchGenre=action&page=2` la dirección URL es la cadena de consulta, `?searchGenre=action&page=2`.
- **`Request.Cookies`**: colección que da acceso a las cookies que el navegador ha enviado.

Ejemplo:

```
@{
    var db = Database.Open("WebPagesMovies");
    var selectCommand = "SELECT * FROM Movies";
    var searchTerm = "";

    if(!Request.QueryString["searchGenre"].IsEmpty() ) {
        selectCommand = "SELECT * FROM Movies WHERE Genre = @0";
        searchTerm = Request.QueryString["searchGenre"]; }

    if(!Request.QueryString["searchTitle"].IsEmpty() ) {
        selectCommand = "SELECT * FROM Movies WHERE Title LIKE @0";
        searchTerm = "%" + Request["searchTitle"] + "%";}

    var selectedData = db.Query(selectCommand, searchTerm);
    var grid = new WebGrid(source: selectedData,
                           defaultSort: "Genre", rowsPerPage:3);
}
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" /><title>Movies</title>
    <style type="text/css">
      .grid { margin: 4px; border-collapse: collapse; width: 600px; }
      .grid th, .grid td { border: 1px solid #C0C0C0; padding: 5px; }
      .head { background-color: #E8E8E8; font-weight: bold; color: #FFF;
      .alt { background-color: #E8E8E8; color: #000; }
    </style>
  </head>
  <body>
    <h1>Movies</h1>
      <form method="get">
        <div>
             <label for="searchGenre">Genre to look for:</label>
             <input type="text" name="searchGenre"
                    value="@Request.QueryString["searchGenre"]" />
             <input type="Submit" value="Search Genre" /><br/>
        (Leave blank to list all movies.)<br/>
        </div>

        <div>
             <label for="SearchTitle">Movie title contains
                     the following:</label>
             <input type="text" name="searchTitle"
                           value="@Request.QueryString["searchTitle"]" />
          <input type="Submit" value="Search Title" /><br/>
        </div>
      </form>

    <div>
      @grid.GetHtml(
        tableStyle: "grid",
        headerStyle: "head",
        alternatingRowStyle: "alt",
        columns: grid.Columns(
          grid.Column("Title"),
          grid.Column("Genre"),
          grid.Column("Year")
        )
      )
    </div>
  </body>
</html>
```

WebMatrix es un conjunto de herramientas que proporciona una manera sencilla de crear sitios web para Microsoft Windows. Incluye IIS Express, ASP.NET y SQL Server Compact.

Capa de servicios

En esta capa tenemos los siguientes componentes:

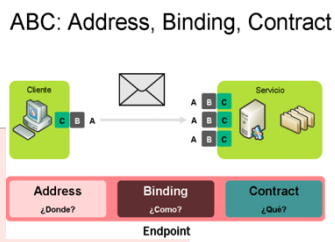


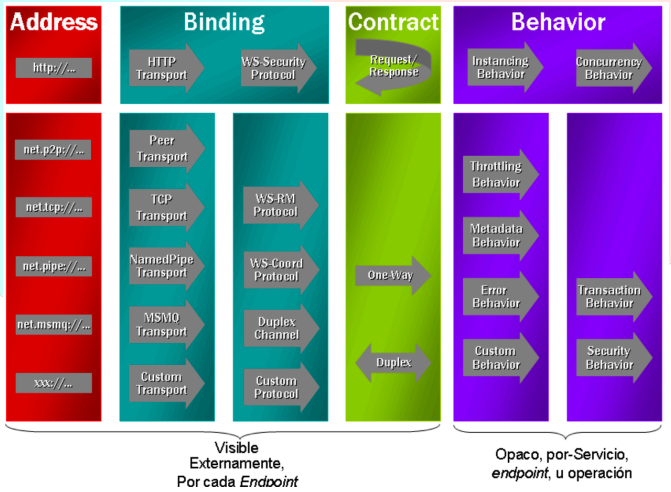


- Interfaz de servicios: es como un servicio web, expone los servicios para que otro servicio web lo use. Son elementos tipo fachada que controlan los servicios de asignación y transformación para permitir la comunicación con un servicio. Implementados mediante **WCF** (Windows Communication Foundation), los servicios web pueden ser SOAP o REST.

  Las siglas "ABC" son claves para WCF porque coinciden con aspectos básicos de cómo están compuestos los End-Points de un Servicio WCF. Los End-Points son básicamente los extremos en las comunicaciones basadas en WCF y por lo tanto también los puntos de entrada a los servicios. Un End-Point es internamente bastante complejo pues ofrece diferentes posibilidades de comunicación, direccionamiento, etc. Un EndPoint está compuesto por "ABC", es decir:

  - "A" para "Address" (Dirección): ***¿Dónde está el servicio situado?*** La dirección se proporciona para los protocolos HTTP, TCP, MSMQ.
  - "B" para "Binding" (Enlace): ***¿Cómo hablo con el servicio?*** El enlace especifica cómo acceder al servicio, definiendo el protocolo de transporte empleado, la codificación de los mensajes y los protocolos WS-Security utilizados.
  - "C" para "Contract" (Contrato): ***¿Qué me ofrece el servicio?*** Es la interfaz al exterior del servicio, especificándose los métodos, tipos y operaciones a exponer.



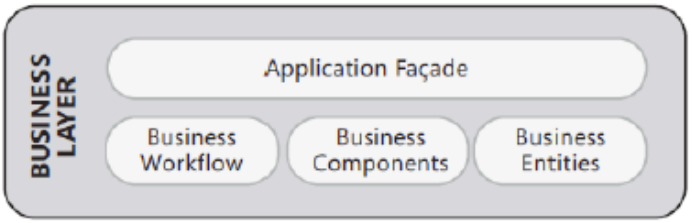


  Conviene resaltar **gRPC**, un marco alternativo a WCF basado en HTTP/2 que ofrece sobre WCF un mayor rendimiento y escalabilidad.

- Intercambio de mensajes.

Capa empresarial

En esta capa tenemos los siguientes componentes:



- Fachada de la aplicación (Application Facade): es un componente opcional que se utiliza cuando la comunicación con los componentes de lógica de presentación requiere de la composición de funcionalidad de varios componentes empresariales.

- Flujos de trabajo empresariales (workflows): **WF** (Windows Workflow Foundation).

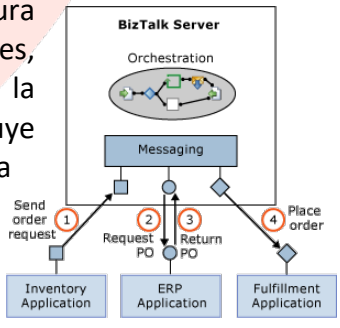
  WF genera el nivel de abstracción para desarrollar aplicaciones interactivas de ejecución prolongada. Las unidades de trabajo se encapsulan como actividades. Las actividades se ejecutan en un entorno que proporcione los medios para el control de flujo, el control de excepciones, la propagación de errores, la persistencia de los datos de estado, la carga y descarga de flujos de trabajo en progreso de la memoria, el seguimiento y el flujo de la transacción.

- Componentes de Negocio: implementan las reglas de negocio.

- Entidades de Negocio: representan los datos que se pasan entre los componentes. Las entidades empresariales se pueden implementar en .NET mediante ADO.NET Entity Framework. Permite utilizar clases **POCO**[2] (Plain Old ClrObjects), que son clases del modelo de dominio que ignoran la persistencia.

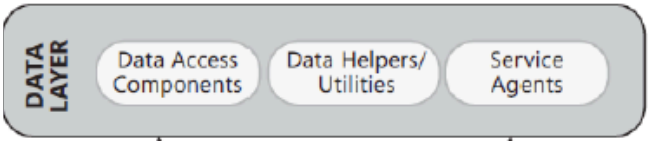
Servidores empresariales:

Microsoft BizTalk Server permite conectar software diverso. Es una arquitectura de publicación y suscripción que usa adaptadores para recibir y enviar mensajes, implementa procesos empresariales a través de la orquestación e incluye la administración y el seguimiento de estas distintas partes. BizTalk Server incluye también la administración de socios comerciales para mensajería de negocio a negocio, alta disponibilidad para maximizar el tiempo de actividad, una plataforma de desarrollo para crear sus propios componentes, una consola de administración para administrar los artefactos y la supervisión de la actividad empresarial para administrar agregaciones, alertas y perfiles.



[2] Es una estructura de datos de .NET que solo contiene propiedades o campos públicos. Un POCO no debe contener ningún otro miembro, como métodos, eventos o delegados. Un POCO no hereda de otra clase ni implementa una interfaz. Es habitual que los POCO se usen con la serialización.

3. Persistencia



En la capa de acceso a datos o de persistencia acedemos a los datos de 3 formas diferentes:

- **Agentes de Servicios**: permiten a los componentes empresariales acceder a un servicio externo encapsulando el acceso a un servicio.
- **ADO.NET**: mediante un conjunto de clases.
- **Entity Framework**: mediante el mapeo objeto/relacional.

ADO.NET (ActiveX Data Object.NET) es un conjunto de clases que exponen servicios de acceso a datos. Proporciona acceso coherente a orígenes de datos como SQL Server y XML, así como a orígenes de datos expuestos mediante OLE DB y ODBC.

ADO.NET separa el acceso a datos de la manipulación de datos y crea componentes discretos que se pueden utilizar por separado o conjuntamente.

Los dos **componentes principales** de ADO.NET para tener acceso a los datos y manipularlos son:

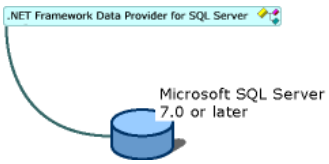
- Los proveedores de datos .NET Framework.
- El `DataSet`.

(1) Proveedores de datos .NET Framework (Data provider)

Los **proveedores de datos .NET Framework** son componentes diseñados explícitamente para:

- la manipulación de datos y
- el acceso rápido a datos de solo lectura y solo avance.

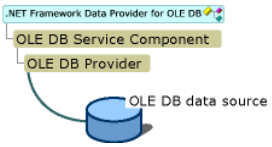
Podemos utilizar los siguientes proveedores de datos .NET Framework según el origen de datos:



- **SQL Server**: usa su propio protocolo para comunicarse con SQL Server. Es ligero y funciona bien porque está optimizado para tener acceso a una SQL Server directamente sin agregar una capa de conectividad de base de datos (ODBC) OLE DB o abierta.

  Para hacer uso en una aplicación C# referenciamos el espacio de nombres:

```
using System.Data.SqlClient;
```



- **OLE DB**: se comunica con un origen de datos de OLE DB a través del componente de servicio OLE DB, que proporciona agrupación de conexiones y servicios de transacción, y el proveedor de OLE DB para el origen de datos.

  Para hacer uso en una aplicación C#:

  ```
  using System.Data.OleDbd;
  ```

- **ODBC**: utiliza el administrador de controladores ODBC (DM) nativo para habilitar el acceso a los datos.

  Para hacer uso en una aplicación C# hacemos referencia al espacio de nombres:

  ```
  using System.Data.Odbc;
  ```

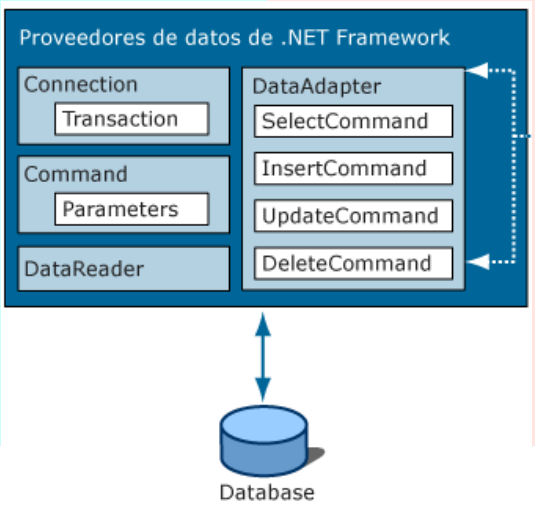
- **Oracle**: permite el acceso a los datos de los orígenes de datos de Oracle a través del software de conectividad de cliente de Oracle.

  Para hacer uso en una aplicación C# hacemos referencia al espacio de nombres:

  ```
  using System.Data.OracleClient;
  ```

- **EntityClient**: proveedor de datos especial que se usa para obtener acceso a datos basándose en un Entity Data Model (EDM). A diferencia de otros proveedores de datos .NET Framework, no interactúa directamente con ningún origen de datos. En su lugar, usa Entity SQL para comunicarse con el proveedor de datos subyacente.

Los **objetos principales** de los proveedores de datos son:



- El objeto `Connection` proporciona conectividad a un origen de datos específico. Se requiere especificar la información de autenticación necesaria en una cadena de conexión. Cada proveedor de datos concreto incluye un objeto `DbConnection`:
  - SQL Server: `SqlConnection`.
  - OLE DB: `OleDbConnection`.
  - ODBC: `OdbcConnection`.

- Oracle: `OracleConnection`.
- EntityClient: `EntityConnection`.

Ejemplo:

```
OdbcConnection connection = new OdbcConnection(connectionString);
```

Para abrir la conexión invocamos el método `Open` de la conexión.

- El objeto **`Command`** permite tener acceso a comandos de base de datos para devolver datos, modificar datos, ejecutar procedimientos almacenados y enviar o recuperar información sobre parámetros. Lo utilizamos una vez la conexión esté abierta.

- **`DataReader`** lee un flujo de datos de solo avance y **solo lectura hacia delante** de alto rendimiento desde el origen de datos. Los resultados se devuelven cuando se ejecuta la consulta y se almacenan en el búfer de red en el cliente hasta que se solicitan mediante el método `Read` del `DataReader`. El uso de `DataReader` puede aumentar el rendimiento de la aplicación al recuperar los datos tan pronto como estén disponibles y, de forma predeterminada, almacenar solo una fila a la vez en la memoria, lo que reduce la sobrecarga del sistema.

Ejemplo:

```
using System;
using System.Data;
using System.Data.SqlClient;

class Program {
    static void Main() {
        string connectionString =
            "Data Source=(local);Initial Catalog=Northwind;"
            + "Integrated Security=true";

        // Provide the query string with a parameter placeholder.
        string queryString =
            "SELECT ProductID, UnitPrice, ProductName from dbo.products "
                + "WHERE UnitPrice > @pricePoint "
                + "ORDER BY UnitPrice DESC;";

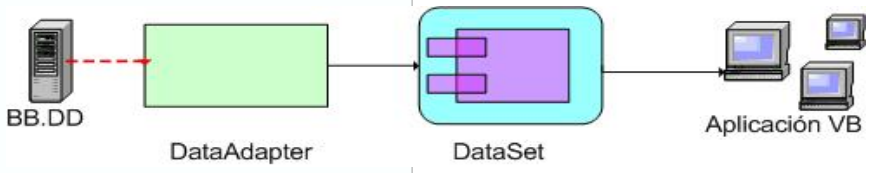
        // Specify the parameter value.
        int paramValue = 5;

        // Create and open the connection in a using block. This
        // ensures that all resources will be closed and disposed
        // when the code exits.
        using (SqlConnection connection = new SqlConnection(connectionString)){
            // Create the Command and Parameter objects.
            SqlCommand command = new SqlCommand(queryString, connection);
            command.Parameters.AddWithValue("@pricePoint", paramValue);

            // Open the connection in a try/catch block.
            // Create and execute the DataReader, writing the result
            // set to the console window.
            try {
```

```
                connection.Open();
                SqlDataReader reader = command.ExecuteReader();
                while (reader.Read()){
                    Console.WriteLine("\t{0}\t{1}\t{2}",
                        reader[0], reader[1], reader[2]);}
                reader.Close();
            }
            catch (Exception ex) {Console.WriteLine(ex.Message);}
            Console.ReadLine();
        }
    }
```

- El objeto **DataAdapter** proporciona el puente entre el objeto DataSet y el origen de datos. Accedemos a los datos de forma **desconectada**. DataAdapter utiliza objetos Command para ejecutar comandos SQL en el origen de datos tanto para cargar el DataSet con datos como para consolidar en el origen de datos los cambios aplicados a los datos incluidos en el DataSet.



El método **Fill** de DataAdapter se usa para rellenar un objeto DataSet con los resultados del elemento SelectCommand de DataAdapter. Fill toma como argumentos un elemento DataSet que se debe rellenar y un objeto DataTable.

Ejemplo:

```
// connection es un objeto SqlConnection
string queryString = "SELECT CustomerID, CompanyName FROM dbo.Customers";
SqlDataAdapter adapter = new SqlDataAdapter(queryString, connection);

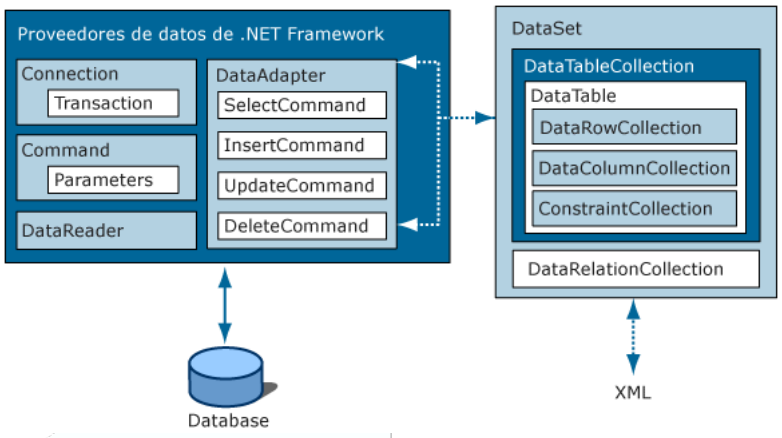
DataSet customers = new DataSet();
adapter.Fill(customers, "Customers");
```

Ejemplo:

```
private static DataSet SelectRows(DataSet dataset,
    string connectionString,string queryString) {
    using (SqlConnection connection = new SqlConnection(connectionString)) {
        SqlDataAdapter adapter = new SqlDataAdapter();
        adapter.SelectCommand = new SqlCommand(queryString, connection);
        adapter.Fill(dataset);
        return dataset; }
}
```

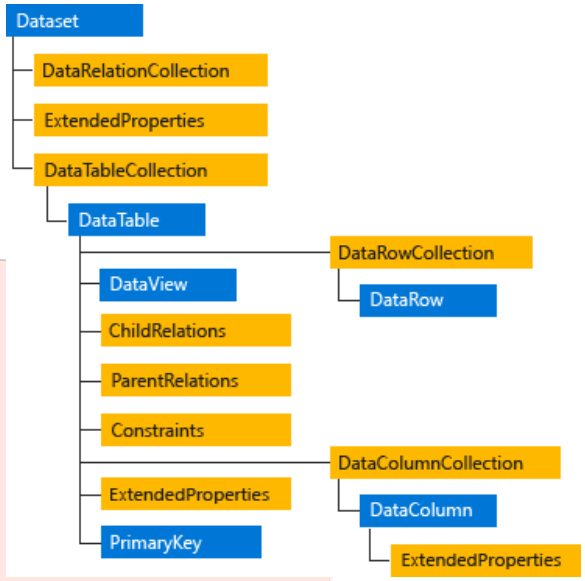
(2) DataSet

DataSet de ADO.NET representa una caché de datos en memoria y está expresamente diseñado para el acceso a datos independientemente del origen de datos. Como resultado, se puede utilizar con múltiples y distintos orígenes de datos, con datos XML o para administrar datos locales de la aplicación.



`DataSet` contiene una colección de uno o más objetos `DataTable` formados por filas y columnas de datos, así como información sobre claves principales, claves externas, restricciones y de relación relacionada con los datos incluidos en los objetos `DataTable`.

En el diagrama siguiente se ilustra el modelo de objetos de un `DataSet`:



- Un **`DataSet`** representa una colección de datos y contiene un `DataRelationCollection` y un `DataTableCollection`.
- Un **`DataRelationCollection`** contiene una colección de relaciones entre tablas.
- Un **`DataRelation`** representa una relación entre tablas.
- Un **`DataTableCollection`** contiene una colección de tablas (uno o varios `DataTable`).
- Un **`DataTable`** representa una tabla.
- Un **`DataColumn`** representa una columna.
- Un **`DataRow`** representa una fila.

Datos relevantes de un DataSet:

- Es sensible a mayúsculas y minúsculas en relación con el nombre de las tablas o de las relaciones.
- El constructor **`DataSet()`** inicializa una nueva instancia de la clase DataSet. Si no se indica como parámetro el nombre del DataSet se le asigna por defecto "`NewDataSet`".
- El método **`AcceptChanges()`** confirma todos los cambios realizados en este DataSet desde que se ha cargado o desde la última vez que se ha llamado a `AcceptChanges()`.
- El método **`GetChanges()`** obtiene una copia del objeto DataSet que contiene todos los cambios que se le han realizado desde la última vez que se cargó o desde que se llamó a `AcceptChanges()`.
- El método **`Clear()`** borra cualquier dato de DataSet mediante el procedimiento de quitar todas las filas de todas las tablas.

- El método **`Clone()`** copia la estructura de DataSet, incluidos todos los esquemas, relaciones y restricciones de DataTable, pero no copia ningún dato.
- El método **`Copy()`** copia la estructura y los datos de DataSet.
- El método **`Load()`** rellena un objeto DataSet con valores de un origen de datos.
- El método **`Reset()`** borra todas las tablas y quita todas las relaciones, restricciones externas y tablas de DataSet.

Ejemplo:

```
using System;
using System.Data;
using System.Data.SqlClient;
namespace Microsoft.AdoNet.DataSetDemo {
    class NorthwindDataSet {
        static void Main() {
            string connectionString = GetConnectionString();
            ConnectToData(connectionString);
        }

        private static void ConnectToData(string connectionString) {
            //Create a SqlConnection to the Northwind database.
            using (SqlConnection connection = new SqlConnection(connectionString))
            {
                //Create a SqlDataAdapter for the Suppliers table.
                SqlDataAdapter adapter = new SqlDataAdapter();

                // A table mapping names the DataTable.
                adapter.TableMappings.Add("Table", "Suppliers");

                // Open the connection.
                connection.Open();
                Console.WriteLine("The SqlConnection is open.");

                // Create a SqlCommand to retrieve Suppliers data.
                SqlCommand command = new SqlCommand(
                    "SELECT SupplierID, CompanyName FROM dbo.Suppliers;",
                    connection);
                command.CommandType = CommandType.Text;

                // Set the SqlDataAdapter's SelectCommand.
                adapter.SelectCommand = command;

                // Fill the DataSet.
                DataSet dataSet = new DataSet("Suppliers");
                adapter.Fill(dataSet);

                // Create a second Adapter and Command to get
                // the Products table, a child table of Suppliers.
                SqlDataAdapter productsAdapter = new SqlDataAdapter();
                productsAdapter.TableMappings.Add("Table", "Products");

                SqlCommand productsCommand = new SqlCommand(
                    "SELECT ProductID, SupplierID FROM dbo.Products;",
                    connection);
                productsAdapter.SelectCommand = productsCommand;
```

```
            // Fill the DataSet.
            productsAdapter.Fill(dataSet);

            // Close the connection.
            connection.Close();
            Console.WriteLine("The SqlConnection is closed.");

            // Create a DataRelation to link the two tables
            // based on the SupplierID.
            DataColumn parentColumn =
                dataSet.Tables["Suppliers"].Columns["SupplierID"];
            DataColumn childColumn =
                dataSet.Tables["Products"].Columns["SupplierID"];
            DataRelation relation =
                new System.Data.DataRelation("SuppliersProducts",
                parentColumn, childColumn);
            dataSet.Relations.Add(relation);
            Console.WriteLine(
                "The {0} DataRelation has been created.",
                relation.RelationName);
        }
    }

    static private string GetConnectionString() {
        // To avoid storing the connection string in your code,
        // you can retrieve it from a configuration file.
        return "Data Source=(local);Initial Catalog=Northwind;"
            + "Integrated Security=SSPI";
    }
  }
}
```

LINQ (Language-Integrated Query)

Muchos desarrolladores deben usar dos (o más) lenguajes de programación: un lenguaje de alto nivel para los niveles de presentación y lógica (C#) y un lenguaje de consulta para interactuar con la base de datos (como TRANSACT-SQL). Esto requiere que el programador tenga conocimientos de varios lenguajes para ser efectivo y también causa discrepancias de "idiomas" en el entorno de desarrollo.

```
// Northwnd inherits from System.Data.Linq.DataContext.
Northwnd nw = new Northwnd(@"northwnd.mdf");
// or, if you are not using SQL Server Express
// Northwnd nw = new Northwnd("Database=Northwind;Server=

var companyNameQuery =
    from cust in nw.Customers
    where cust.City == "London"
    select cust.CompanyName;

foreach (var customer in companyNameQuery)
{
    Console.WriteLine(customer);
}
```

Language-Integrated Query (LINQ) es el nombre de un conjunto de tecnologías basadas en la integración de capacidades de consulta (lenguaje integrado) a distintas fuentes de datos (objetos, BD relacionales y XMLs) directamente en C# y Visual Basic. Permite, por tanto, crear consultas en el código fuente sin tener que usar un lenguaje de consulta independiente.

```
// Specify the data source.
int[] scores = { 97, 92, 81, 60 };

// Define the query expression.
IEnumerable<int> scoreQuery =
    from score in scores
    where score > 80
    select score;

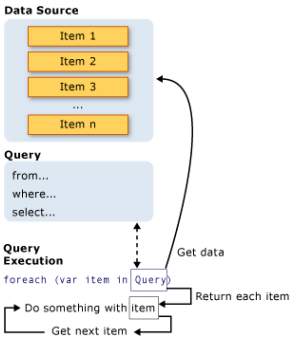
// Execute the query.
foreach (int i in scoreQuery)
{
    Console.Write(i + " ");
}

// Output: 97 92 81
```

LINQ supera los siguientes problemas:

- Tradicionalmente, las consultas con datos se expresaban como cadenas simples sin comprobación de tipos en tiempo de compilación ni compatibilidad.
- Es necesario aprender un lenguaje de consultas diferente para cada tipo de origen de datos: bases de datos SQL, documentos XML y servicios web, entre otros.

Todas las operaciones de consulta LINQ constan de 3 acciones distintas:



- Obtener el origen de datos.
- Crear la consulta.
- Ejecutar la consulta: se ejecuta en una instrucción `foreach` la cual requiere `IEnumerable` o bien `IEnumerable<T>`. Los tipos compatibles con `IEnumerable<T>` o una interfaz derivada, como la interfaz genérica `IQueryable<T>`, se denominan tipos consultables.

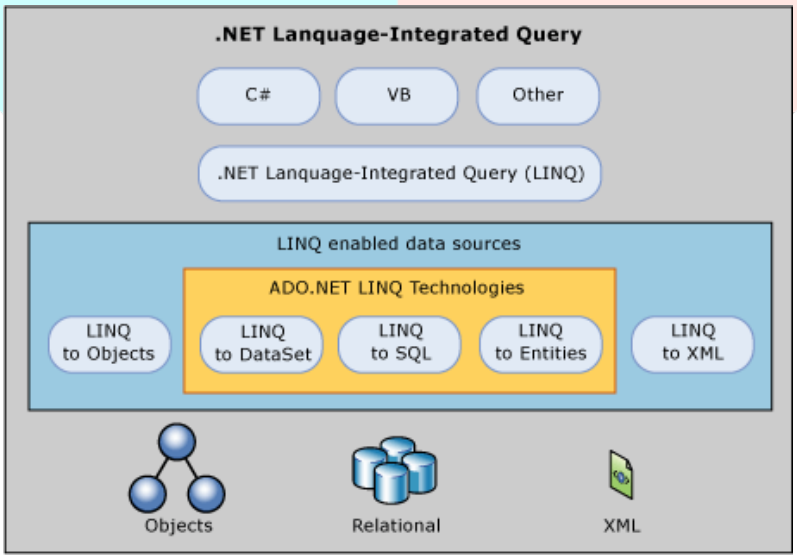
Ejemplo: las 3 acciones de una consulta LINQ.

```
// The Three Parts of a LINQ Query:
// 1. Data source.
Int[] numbers = [ 0, 1, 2, 3, 4, 5, 6 ];

// 2. Query creation.
// numQuery is an Ienumerable<int>
var numQuery =
    from num in numbers
    where (num % 2) == 0
    select num;

// 3. Query execution.
Foreach (int num in numQuery) {
    Console.Write("{0,1} ", num);
}
```

Se muestra a continuación la visión general de cómo se relacionan las tecnologías ADO.NET LINQ con lenguajes de programación de alto nivel:



LINQ proporciona una consulta coherente para:

- Objetos (**LINQ to Objects**): acceso a objetos de .NET.
- **LINQ to ADO.NET**:
  - DataSet (**LINQ to DataSet**): acceso a datos mediante DataSet, facilitando y acelerando la consulta sobre los datos almacenados en caché en un objeto DataSet.
  - Bases de datos relacionales (**LINQ to SQL**): uso de LINQ directamente sobre el esquema de la base de datos mediante un modelo de objetos para manejar el modelo de datos de la base de datos.
  - Entidades (**LINQ to Entities**): las consultas en LINQ se realizan contra un modelo conceptual de Entity Framework.
- XML (**LINQ to XML**): acceso a documentos XML.

Parallel LINQ (**PLINQ**) es una variante de LINQ que permite la ejecución paralela de consultas.

Entity Framework es una mapeador objeto/relacional (ORM) que permite trabajar a un nivel mayor de abstracción con una base de datos mediante objetos .NET. Para ello, disponemos de:

- **Entity Framework 6** (**EF6**) para .NET Framework: es un ORM diseñado para .NET Framework, pero compatible con .NET Core. EF6 es un producto estable y compatible, pero ya no se desarrolla activamente.

- **Entity Framework Core** (**EF Core**) para .NET Core: es un ORM de código abierto y multiplataforma para .NET, más moderno, ligero y extensible que EF6. Admite consultas LINQ, seguimiento de cambios, actualizaciones y migraciones de esquemas. EF Core funciona con, entre otras, SQL Server o SQL Azure, SQLite, Azure Cosmos DB, MySQL y PostgreSQL.

  Ejemplo:

  MODELO

```
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;

public class BloggingContext : DbContext {
    public DbSet<Blog> Blogs { get; set; }
    public DbSet<Post> Posts { get; set; }

    public string DbPath { get; }

    public BloggingContext()  {
        var folder = Environment.SpecialFolder.LocalApplicationData;
        var path = Environment.GetFolderPath(folder);
        DbPath = System.IO.Path.Join(path, "blogging.db");
    }
```

```
    // The following configures EF to create a Sqlite database file in the
    // special "local" folder for your platform.
    protected override void OnConfiguring(DbContextOptionsBuilder options)
        => options.UseSqlite($"Data Source={DbPath}");
}

public class Blog {
    public int BlogId { get; set; }
    public string Url { get; set; }
    public List<Post> Posts { get; } = new();
}

public class Post {
    public int PostId { get; set; }
    public string Title { get; set; }
    public string Content { get; set; }
    public int BlogId { get; set; }
    public Blog Blog { get; set; }
}
```

PROGRAMA

```
using System;
using System.Linq;

using var db = new BloggingContext();

// Note: This sample requires the database to be created before running.
Console.WriteLine($"Database path: {db.DbPath}.");

// Create
Console.WriteLine("Inserting a new blog");
db.Add(new Blog { Url = "http://blogs.msdn.com/adonet" });
db.SaveChanges();

// Read
Console.WriteLine("Querying for a blog");
var blog = db.Blogs
    .OrderBy(b => b.BlogId)
    .First();

// Update
Console.WriteLine("Updating the blog and adding a post");
blog.Url = "https://devblogs.microsoft.com/dotnet";
blog.Posts.Add(
    new Post { Title = "Hello World", Content = "I wrote an app using EF Core!"
});
db.SaveChanges();

// Delete
Console.WriteLine("Delete the blog");
db.Remove(blog);
db.SaveChanges();
```

4. Seguridad

La seguridad en .NET incluye las siguientes áreas:

- **Seguridad de tipos**: los programas de tipos seguros hacen referencia únicamente a la memoria que ha sido asignada para su uso, y acceden a objetos únicamente a través de sus interfaces expuestas. Desde el punto de vista de seguridad, al hacer referencia únicamente a la memoria asignada se permite que múltiples objetos compartan de manera segura un espacio de dirección único. Al acceder a los objetos únicamente a través de sus interfaces expuestas garantiza que las revisiones de seguridad relacionadas con interfaces específicas no sean evitadas. CLR garantiza la seguridad de tipos al requerir la verificación antes de permitir ejecutarse.

- **Evidencia**: existen únicamente dos formas para que un código sea ejecutable. La primera, a través del cargador de clases, la segunda a través de los servicios de interoperabilidad. Estos dos servicios son proporcionados por CLR y son parte del perímetro de seguridad. Por ejemplo, el cargador de clases mantiene información acerca de la fuente para cada implementación que carga. Por lo tanto, el cargador de clases puede proporcionar de manera confiable alguna evidencia sobre la que basar la identidad del código. La evidencia puede incluir información como zona de Internet y sitio del que el código fue originado, su nombre compartido, y la identidad del publicador. Utilizando esta información, la política de seguridad puede controlar los privilegios proveídos a una aplicación especifica o conjunto de aplicaciones relacionadas.

- **Permisos de recurso**: estos permisos están relacionados con la información o recursos de comunicaciones dentro del sistema, como archivos, ventanas, el portapapeles, canales de red, almacenamiento privado de aplicaciones… Estos tipos de permiso se pueden utilizar para determinar si el código tiene derecho de acceder a recursos específicos ya sea en tiempo de carga o de ejecución.

- **Permisos de identificación**: estos permisos se basan en la evidencia relacionada con un ensamblado determinado (sitio, zona, nombre compartido y publicador). Permiten controlar el acceso a métodos de clase basados en esta información. Los permisos de identificación operan de la misma forma que los permisos a recursos. Si los invocadores tienen el atributo de identidad requerido, la llamada tiene éxito; de otra forma se lanza una excepción de seguridad.

- **Seguridad declarativa**: este mecanismo permite insertar verificaciones de seguridad de acceso de código en el código declarando las clases, campos y métodos. Estas declaraciones son codificadas en los metadatos del ensamblado y son forzadas por el sistema de seguridad .NET. Al utilizar estas declaraciones, se puede solicitar que se realicen verificaciones ya sea en tiempo de carga o de ejecución. Aunque son más limitadas, las verificaciones de tiempo de carga son más eficaces ya que sólo son revisadas una vez durante la ejecución, en contraposición a que sean revisadas en cada llamada a un método.

- **Seguridad imperativa**: implementada por el desarrollador de manera programática dentro de un método. Son forzados exactamente de la misma forma que las revisiones

en tiempo de ejecución declarativas. El beneficio de utilizarlas es que puede soportar la determinación dinámica de permisos específicos que son necesarios en un contexto de ejecución determinado.

- **Seguridad basada en políticas**: permitir el uso efectivo de seguridad de acceso de código requiere un sistema basado en políticas. La política le permite declarar específicamente lo que puede permitirse al código, con base en su punto de origen y otra información de identificación. Al utilizar esta política, automáticamente se otorgan los privilegios correspondientes para el código sin requerir interacción alguna con el usuario. Es muy probable que .NET incluya una política predeterminada que refleja los diferentes niveles de confianza que comúnmente le coloca a su código de sistema local, al código de servidores confiables conocidos, y al código de sitos de Internet no confiables. Se puede refinar esta política si es necesario.

- **Seguridad basada en roles**: es común para los desarrolladores tomar decisiones de autorización basadas en la identidad o roles asociados con el contexto de ejecución. Con frecuencia se ve esto en sistemas financieros o de negocios como medios para forzar políticas. Por ejemplo, pueden poner límites en el valor de una transacción, dependiendo del rol del usuario que hace la solicitud. Los empleados deben poder procesar transacciones hasta un valor determinado, los supervisores hasta un nivel más alto, y los gerentes en un nivel aún más alto. .NET proporciona servicios para permitir que las aplicaciones incorporen fácilmente dicha lógica de manera que sea independiente de la plataforma y escalable para Internet. Los mecanismos .NET se desarrollan alrededor de la noción de identidades y principales.

  El objeto **Identity** encapsula información sobre el usuario o la entidad que se está validando. En su nivel más básico, los objetos Identity contienen un nombre y un tipo de autenticación. Las identidades encapsulan nombres de identidades que entiende la aplicación. Esto puede relacionarse con la noción de cuentas del sistema operativo, pero también pueden ser definidos con la aplicación.

  El objeto **Principal** representa el contexto de seguridad bajo el cual se ejecuta código. Las aplicaciones que implementan seguridad basada en roles conceden derechos tomando como base el rol asociado a un objeto Principal. El correspondiente principal encapsula la identidad, junto con la información del rol. Esto puede o no relacionarse con una noción del sistema operativo como los grupos.

  En operación, un proveedor de autenticación confiable típicamente computa la información principal con base en las credenciales relacionadas con una petición determinada (HTTP PUT, llamada a procedimiento remoto…). Así pues, el principal se adjunta al contexto de ejecución, poniéndolo disponible al código de la aplicación para verificar el rol o la información de identidad antes de tomar una acción determinada.

- **Seguridad de ejecuciones remotas**: .NET proporciona soporte para la invocación de objetos remotos que pueden expandir AppDomains. AppDomain es una aplicación lógica y múltiples AppDomains se pueden ejecutar en un solo proceso Win32. Al cruzar los límites de la máquina, los aspectos de seguridad tales como autenticación, autorización,

confidencialidad e integridad son críticos, de manera que .NET proporciona soporte para estos mecanismos a fin de que sea compatible con los protocolos de red existentes, y los integra firmemente en la infraestructura de ejecuciones remotas. Esto es más simple para incorporar estas características en sus aplicaciones. Los mecanismos de autenticación son adecuados para la identificación de usuarios, así como aplicaciones o entidades de negocios. Una infraestructura de identidad basada en claves se proporciona para administrar tales entidades e integrarlas con los protocolos de autenticación basados en claves. Los servicios de confidencialidad e integridad aprovechan las técnicas criptográficas. Es posible utilizar mecanismos de punto a punto tales como TLS/SSL e IPSec. Tecnologías de encriptación de la capa de la aplicación, integridad y de firma digital en el nivel de mensajería también son soportadas por el entorno .NET.

- **Criptografía**: .NET proporciona un conjunto de objetos criptográficos que soportan algoritmos bien conocidos y usos comunes incluyendo partición, encriptación y generación de firmas digitales. Estos objetos están diseñados de manera que facilitan la incorporación de estas capacidades básicas en operaciones más complejas, tales como la firma y encriptación de un documento. Los objetos criptográficos son utilizados por .NET para soportar servicios internos, pero también están disponibles para los desarrolladores que necesitan soporte criptográfico.

  La criptografía .NET implementa un patrón extensible de herencia de clases derivadas, situándose en la jerarquía las clases abstractas `SymmetricAlgorithm`, `AsymmetricAlgorithm` o `HashAlgorithm`.

5. Administrador de paquetes

NuGet es un administrador de paquetes para el ecosistema de. NET y es la principal forma que tienen los desarrolladores para detectar y adquirir bibliotecas de código abierto de .NET. NuGet.org, un servicio gratuito proporcionado por Microsoft para hospedar paquetes NuGet, es el host principal para los paquetes NuGet públicos, pero también es posible publicar en los servicios de NuGet personalizados, como MyGet y Azure Artifacts.

Un **paquete NuGet** (*.**nupkg**) es un archivo ZIP que contiene los ensamblados de .NET y los metadatos asociados.

Para la instalación y administración de paquetes NuGet haremos uso de la CLI de `dotnet`:

- **`dotnet add package <paquete>`**: instala la última versión del paquete.

  ```
  dotnet add package Microsoft.EntityFrameworkCore
  ```

- **`dotnet list package`**: lista las referencias y versiones de los paquetes del proyecto.

- **`dotnet remove package <paquete>`**: elimina el paquete.

  ```
  dotnet remove package Newtonsoft.Json
  ```

- **dotnet restore**: restaura los paquetes incluidos en el archivo de proyecto PackageReference.

- **dotnet pack**: empaquete el código en un paquete NuGet.

- **dotnet build**: compila una aplicación de .NET.

- **dotnet exec**: ejecuta una aplicación de .NET.

```
dotnet exec myapp.dll
```

- **dotnet nuget push**: publica un paquete en un servidor NuGet (nuget.org, Azure Artifacts o servidores NuGet de terceros).

- **dotnet nuget delete**: elimina o quita de la lista un paquete de un servidor NuGet (nuget.org, Azure Artifacts o servidores NuGet de terceros).

- **dotnet nuget verify**: comprueba un paquete de NuGet firmado.

- **dotnet new nugetconfig**: crea un archivo nuget.config para configurar el comportamiento de NuGet.

También se puede administrar paquetes NuGet con la CLI de NuGet:

- **nuget install <paquete>**: instala la última versión del paquete.

```
nuget install package iText
```

- **nuget list**: lista los paquetes instalados del proyecto.

- **nuget update**: actualiza todos los paquetes del proyecto.

- **nuget restore**: restaura los paquetes incluidos en el archivo de proyecto PackageReference o packages.config.

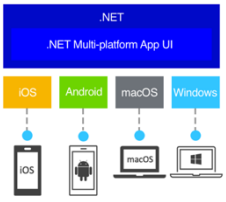
6. Frameworks y otras APIs

Frameworks

- **Blazor**: plataforma de trabajo para la creación de interfaces de usuario web interactivas del lado cliente con .NET.

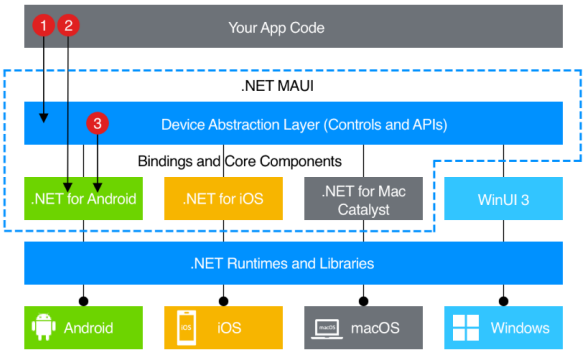
- **.NET Aspire**: framework que facilita la creación, gestión y despliegue de aplicaciones nativas en la nube.

- **.NET MAUI (Multi-platform App UI)**: framework multiplataforma de código abierto para crear aplicaciones móviles y de escritorio nativas con C# y XAML que



se pueden ejecutar en Android, iOS, macOS y Windows desde una sola base de código compartida. Es la evolución de Xamarin.Forms.

.NET MAUI proporciona un único marco para compilar las IU para aplicaciones móviles y de escritorio.



En una aplicación MAUI de .NET, se escribe código que interactúa principalmente con los controles .NET MAUI y la capa de API (1). A continuación, esta capa consume directamente las API de plataforma nativas (3). Además, el código de la aplicación puede ejecutar directamente las API de plataforma (2), si es necesario.

- **NHibernate**: framework de persistencia para .NET.

- **PostSharp Framework**: programación de aplicaciones orientadas a aspectos.

Librerías y herramientas

- **AutoMapper**: mapeador objeto-objeto basado en convenciones.

- **Fluent NHibernate**: extensión de NHibernate para mapeo entre el modelo y la base de datos sin necesidad de ficheros XML.

- **FluentValidation**: biblioteca para construir reglas de validación.

- **log4Net**: implementa un sistema de log.

- **Lucene**: biblioteca de Apache que implementa un motor de búsqueda de texto para recuperación de información.

- **Managed Extensibility Framework (MEF)**: es una biblioteca para crear aplicaciones ligeras y extensibles.

- **ML.NET**: ofrece el aprendizaje automático en aplicaciones .NET.

- **NLog**: sistema de log de código abierto.

- **NSwag**: librería de código abierto que permite diseñar, documentar y consumir servicios REST.

- **NUnit**: evalúa el funcionamiento de nuestro desarrollo mediante pruebas funcionales de clases y métodos.

- **PostSharp Caching**: mejora el rendimiento de la aplicación.

- **PostSharp Loggin**: extensión para registro que se integra con librerías de log.

- **PostSharp MVVM**: extensión para evitar código repetitivo.

- **PostSharp Threading**: escritura de código seguro para hilos.

- **xUnit**: sistema de soporte para pruebas dirigidas por el desarrollo (TDD).

